Chair of Applied Mechanics
TUM School of Engineering and Design
Technical University of Munich

Roboterdynamik Planning

Module MW0867

Lecture Script - Version 0.03
Summer Semester 2025

Prof. dr.ir. Daniel J. Rixen, Moritz Sattler

Technical University of Munich
Chair of Applied Mechanics

Contents

Title Page **i**

4 Planning **1**
4.1 Environment Modelling . . . 1
4.1.1 Division of Space . . . 2
4.1.2 Simplified Geometries . . . 2
4.2 Discrete Planning . . . 4
4.2.1 Graphs . . . 5
4.2.2 Graph Search . . . 5
4.2.3 Graph Building . . . 7
4.2.4 Probabilistic Roadmaps (PRM) . . . 7
4.2.5 Rapidly-exploring Random Trees (RRT) . . . 8
4.3 Continuous Description of Paths and Trajectories . . . 9
4.3.1 Point-To-Point in Joint Space . . . 11
4.3.2 Multiple Points in Joint Space . . . 12
4.4 Path and Trajectory Optimization . . . 14
4.4.1 Simple Example: Inverse Kinematics . . . 15
4.4.2 Lagrange Multipliers . . . 16
4.4.3 Inequality conditions - KKT conditions . . . 20
4.4.4 Time-local vs. Time-Global Optimization . . . 23
4.4.5 Examples of Trajectory Optimization Strategies . . . 25
4.4.6 Direct Optimization . . . 27
4.4.7 Indirect Optimization . . . 31

Bibliography **35**

Chapter 4

Planning

The goal of motion planning is to find a trajectory from a given initial state of the robot to a goal state or a goal region. On this trajectory, the robot must not collide with the environment or with itself. Furthermore, the trajectory must be feasible for the given robot: There are usually limits on the joint angles and the torques in the joints. Since the relation between the joint trajectory and the torque is defined by the dynamic equations of the robot, we have to consider these differential equations. Alternatively, we could use a more restrictive simplification of the torque limits, such as restricting accelerations and velocities so that the torque limits are satisfied.

$$\boldsymbol{x}_{\text{start}} \rightarrow \boldsymbol{x}_{\text{goal}}$$
$$\text{collision-free}$$
$$\boldsymbol{q}_{\min} \leq \boldsymbol{q} \leq \boldsymbol{q}_{\max}$$
$$\boldsymbol{Q}_{\min} \leq \boldsymbol{Q} \leq \boldsymbol{Q}_{\max}$$

In some applications, we want to reach the goal as fast as possible – in general, the time should at least not be too long. The task of motion planning is often divided into path planning (Bahnplanung) and trajectory planning (Trajektorienplanung). A path consists of all geometric points along which the robot moves. Thus, path planning is about finding a collision-free path through the environment. A trajectory combines a path with time information. It is therefore the evolution of the robot's state variables over time. Thus, trajectory planning can use a pre-planned path and adds the timing information, taking into account the dynamic capabilities of the robot. Planning can be done in the joint space ($\boldsymbol{q}$), in the workspace ($\boldsymbol{w}$), or in a state space ($\boldsymbol{x}$, which also includes velocities). For redundant robots, the joint space not the workspace parameters describe the configuration of the robot. Therefore, in this chapter, we will focus on planning in the joint space. Our configuration space (c-space) is the joint space.

The parts of this chapter are visualized in fig. 4.1. First we will introduce options for modeling the environment ① for motion planning. Then we will learn about discrete planning ②, which we can use e. g. for path planning. To add timing information, the next part introduces trajectory description ③. And at the end of this chapter some basic concepts of optimization are presented. Optimization techniques ③ can be used to smooth and improve the existing result.

4.1 Environment Modelling

During path planning, we have to check collisions between the robot and the environment, and also between different bodies of the robot. Therefore, we need a description

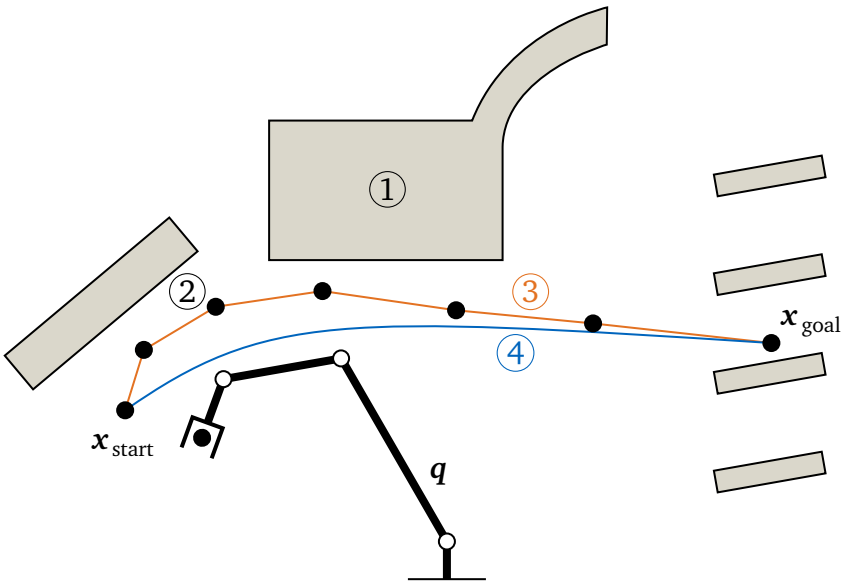


Figure 4.1: Overview Chapter Planning

of the environment that suits for collision checks and distance calculations. The information about the environment can come from a human user if the environment is static. In case of dynamic environments or environments that are unknown to the human user, the information comes from a vision system that is not discussed in this course. Vision systems or CAD programs that are used to build the industrial environment around our robot typically output point clouds or detailed triangle meshes. Collision checks and distance calculations using this detailed data are expensive. As in most planning applications the computation time is dominated by the time spent on collision checking [6], it can be important to reduce this time. The following subsections discuss how to reduce the number of calculations required and how to simplify these calculations.

4.1.1 Division of Space

To reduce the number of calculations, the space can be divided into separate parts. If two objects are not in the same part, they cannot collide. For a distance calculation, only objects that are in the same part are considered (and maybe the neighbors, depending on the maximum distance of interest in the current application). Note that collision checks are generally cheaper than distance calculations.

One possibility to divide the space is to take a cube that covers the entire space of interest, the parent cube. This cube is then subdivided in eight child cubes of the same size. (In 2D, the analogy would be a square divided into four child squares as in fig. 4.2.) Then each non-empty cube is recursively subdivided again and again until some criterion is reached. This results in a tree in which each node has no children or exactly eight children, an octree as visualized in fig. 4.3. (The 2D analog is a quadtree.) The objects that lie in a leave cube are stored in the corresponding leave node of the octree.

4.1.2 Simplified Geometries

The complexity of the distance calculation between to object depends on their representation.

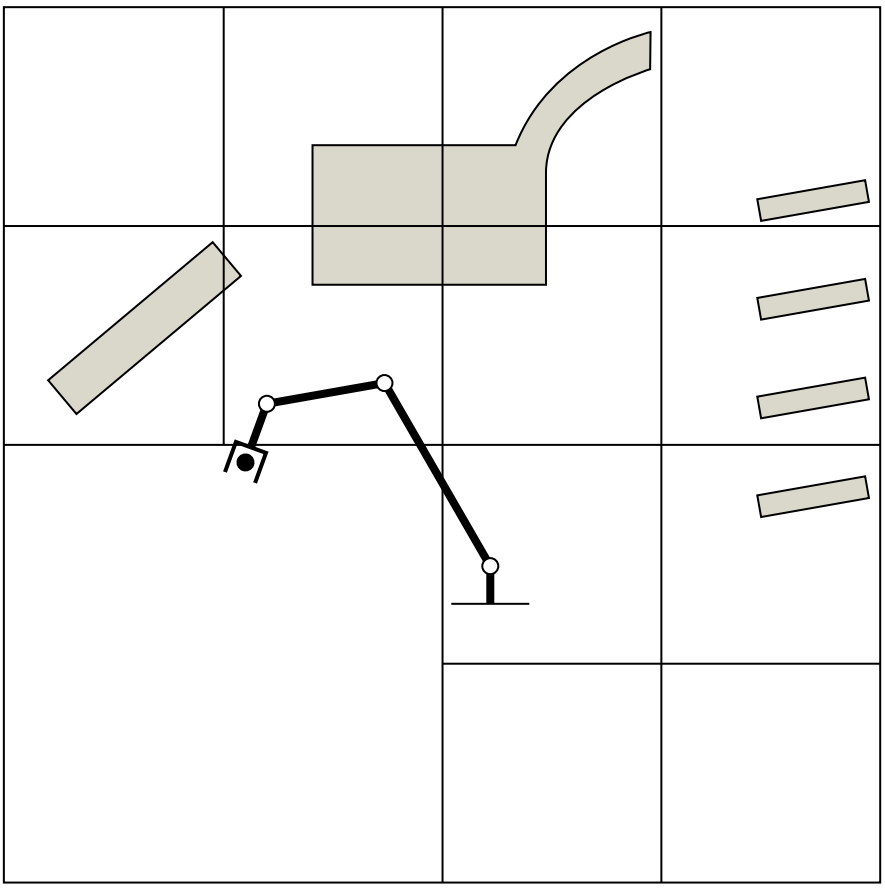


Figure 4.2: Division of Space

Triangle Meshes

are mainly used for very detailed surface descriptions. Such a detailed description may be necessary when the robot interacts with the environment, e.g. when a robot wets a surface with glue. For other motions, the robot should only approach the objects to a certain safety margin. For this purpose, a coarser approximation of the environment is sufficient. One way is to use a coarser mesh. To calculate the distance between two objects, the distance between many triangles of one object and many triangles of the other object must be calculated. This can still be an expensive task. A visualization in our example environment is shown in fig. 4.4.

Bounding Boxes

are rectangular boxes around objects. They are particularly easy to use when they are aligned to a coordinate system as in fig. 4.5. Such a bounding box is simply defined by the minimum and maximum coordinates of all points of the corresponding object. If the coordinate range of two such bounding boxes overlap the two boxes overlap. Of course, the objects do not need to overlap at all if their bounding boxes overlap. Bounding boxes are very coarse and are rather used to check if the collision between two objects has to be checked with a finer representation.

Spheres

are excellent for calculating distances. The distance between two spheres is simply the distance between the two centers minus the two radii. By using multiple spheres, arbitrary

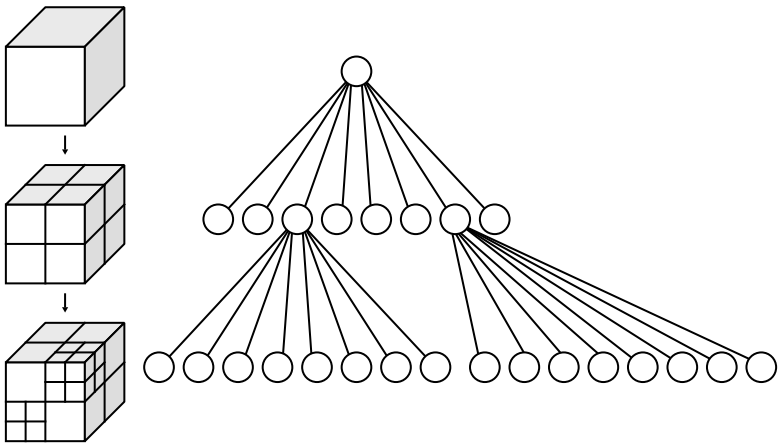


Figure 4.3: Octree (WhiteTimberwolf, CC BY-SA 3.0, via Wikimedia Commons)

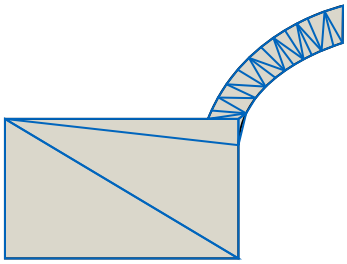


Figure 4.4: Triangle Meshes

shapes of objects can be approximated as shown in fig. 4.6.

Swept-Sphere-Volumes (SSV)

improve the approximation capabilities of spheres while making distance calculations only slightly more complicated. SSVs are based on simple geometries such as points, line segments, triangles, or rectangles. All points within a specified radius around the simple geometry belong to the SSV. This results in a *volume* that can also be generated when a *sphere* with this radius is *swept* along the simple geometry. Some SSV elements are shown in fig. 4.7. The distance calculation is reduced to a distance calculation between the basic geometries. Similar to the distance calculation between spheres, the two radii must be subtracted from this distance to obtain the distance between the two SSVs. [4, 7]

4.2 Discrete Planning

Knowing about environment representations, collision checking, and distance calculation, we now approach the planning. Many popular approaches to search for a feasible path are based on graphs as a discretization of the collision-free configuration space. Therefore, we first introduce graphs and graph search algorithms. Then, two important discrete planning algorithms are explained.

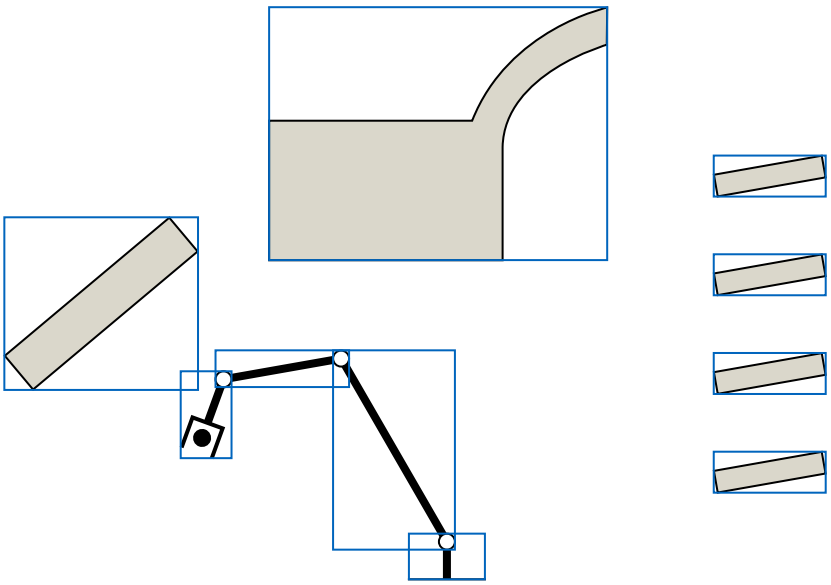


Figure 4.5: Bounding Boxes

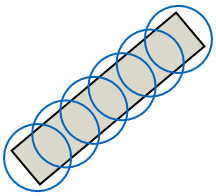


Figure 4.6: Spheres for Geometry Approximation

4.2.1 Graphs

A graph consists of nodes that are connected via edges. You can think of the nodes as cities connected by highways, the edges. The edges can be weighted with the cost of travel, e. g. the distance on the highway between the two cities. Highways can usually be traveled in both directions. A graph can also be directed. This means that the edges can only be taken in one direction, like one-way streets. In robotics, this is important when planning not only in joint space or workspace but in a state space that includes velocities. Then the graph must be directed.

Trees are special graphs that have one root node. All other nodes have exactly one parent node. Thus, a tree has no cycles.

4.2.2 Graph Search

To find a way from one start node to a goal node, there exist several search algorithms.

Dijkstra

The basis of Dijkstra's algorithm is a connected graph with positive edge weights. Dijkstra searches for a path with the lowest cost to get from the start to the goal node. In general, multiple paths can satisfy this condition, where the Dijkstra finds only one of these optimal paths. In the following, C is the cost of a node n equal to the sum of all edge weights g from the starting node to this node. Let C^* be the optimal, i.e., lowest, cost of a node. At the beginning of the algorithm, initialize the starting node with $C^*(n_0) = 0$ and, if other

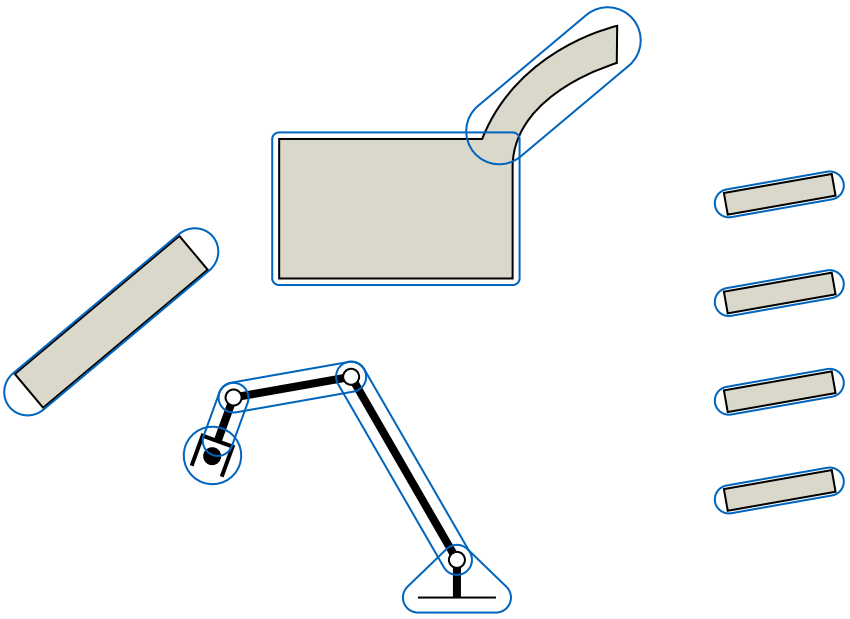


Figure 4.7: Swept Sphere Volumes for Geometry Approximation

nodes are already known, all other nodes with $C(n_i) = \infty$ with $i \neq 0$. Additional nodes can also be added during the search. All nodes with non-optimal cost are in a list that still needs to be processed, the open list. Now select the node n_j with the lowest cost from this list. This lowest cost is its optimal cost. If two nodes have the lowest cost, you can choose which node to consider. Starting from this node n_j, all neighboring nodes n_k are examined. If not all neighbors exist, they are added as new nodes to the open list. For a node n_k the cost over this path is the cost of the predecessor node plus the weight of the edge:

$$C_{\text{new}}(n_k) = C^*(n_j) + C(g_{jk})$$

If the node already existed, this cost is compared to the node's current cost and the lower of the two costs is assigned to it as the new cost.

$$C(n_k) = \min\left(C_{\text{new}}(n_k), C_{\text{current}}(n_k)\right)$$

Since neighbors with the optimal cost can no longer get smaller cost, they do not need to be examined. The algorithm searches until the target node is the cheapest node in the open list. This is because once a node from the open list is considered, i.e. has the lowest cost in the list, these are its optimal cost.

Best-First-Search

The last algorithm used only the cost from the start to the current node. The best-first-search algorithm, on the other hand, does not rely on the previous cost, but always examines the node in the open list whose residual cost is estimated to be the lowest. The residual cost is the cost from the current node to the goal. Here, the open list contains only the nodes that are in the direct neighborhood of the nodes that have already been examined. The estimation is also called heuristic. The Best-First-Search algorithm searches until the goal is reached. In general, it does not find an optimal solution. However, depending on the properties of the heuristic, it usually examines much fewer nodes than Dijkstra. Therefore, with suitable heuristics, this algorithm is characterized by a significantly faster runtime.

A*

A* (A-Star) combines the advantages of Dijkstra and Best-First-Search. In the default variant, it delivers an optimal result, but at least as fast (usually much faster) than the Dijkstra. The improved runtime results from an informed directed search. As with the Best-First-Search, a heuristic is used that directs the search in one direction. The A* also selects the node of the open list with the lowest cost. Here, the cost is the sum of the cost of the path up to here and the heuristic, the estimate of the residual cost.

$$F(n) = C(n) + H(n)$$

Like the Dijkstra, A* searches until the target node is the cheapest node in the open list. The A* returns an optimal result as long as the heuristic underestimates the cost of the remaining edges to the goal. [6]

4.2.3 Graph Building

To use graphs as a representation of the search space, we need to discretize the space. Similar to discretization in other applications, a grid is one possibility. Each $q_{i,\min} \leq q_i \leq q_{i,\max}$ is discretized with some defined Δq_i. There are two difficulties with this approach:

- The number of nodes in the graph grows exponentially with the degrees of freedom.
- Big Δq_i lead to impassable narrow passages between obstacles – small Δq_i to a large number of nodes in the graph.

There are several approaches to solve these issues such as local mesh refinement. However, probabilistic sampling methods to generate nodes are more popular. Two very well-known algorithms are described in the following. The algorithms are presented in a basic version. There are many more sophisticated algorithms based on them.

4.2.4 Probabilistic Roadmaps (PRM)

A Probabilistic Roadmap is used to search several times in a static environment. First, in an offline construction phase, the PRM algorithm builds a graph in the search space as shown in fig. 4.8. Then, in the online query phase, a search algorithm is used to find paths through this graph. The graph remains the same and multiple paths are searched on it.

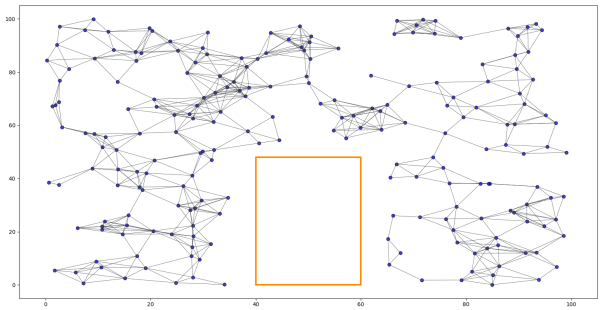


Figure 4.8: Example of PRM (Aabhas Mathur, MIT, via github)

Construction Phase

The algorithm of the construction phase is described in algorithm 1. A random collision-free configuration is added to the list of nodes. All neighbours within a predefined distance are candidates to be directly connected to this new node. For all of these neighbors, the direct edge between a neigbor and the new node is checked for collisions. If the edge is collision-free, it is added to the edges. Especially in the beginning, new nodes may stay unconnected or form small clusters. These steps are repeated for example until a certain number of nodes are added to the graph and preferably no unconnected clusters are left.

Query Phase

In the query phase the start and goal configurations are connected to the graph. Then a graph search algorithm like A* searches a path.

Algorithm 1 Basic PRM Construction Phase

loop
  $\boldsymbol{q}_{\text{random}} \leftarrow$ get random collision-free configuration
  add $\boldsymbol{q}_{\text{random}}$ to the nodes
  $N_c \leftarrow$ get all nodes in a certain distance from $\boldsymbol{q}_{\text{random}}$
  for $\boldsymbol{q}_i \in N_c$ **do**
    check the edge from $\boldsymbol{q}_i$ to $\boldsymbol{q}_{\text{random}}$ for collision
    if no collision on the edge **then**
      add edge
    end if
  end for
end loop

[2]

4.2.5 Rapidly-exploring Random Trees (RRT)

A Rapidly-exploring Random Tree is used to search only once. The graph, here a tree, is build only for a single query. A new tree is build for a next search. A pseudocode version of a simple RRT algorithm is shown in algorithm 2. The algorithm needs to know about the start and the goal. As long as the goal or the goal region is not reached the following steps are repeated:

- Choose a random configuration.
- Search for the closest configuration in the tree to this random configuration.
- Define a new node by starting at the closest configuration in the tree and going a predefined step towards the random configuration.
  (This predefined step gets smaller over the runtime of the algorithm.)
- Before adding this new node to the graph, check the connection between the closest node and the new node for collisions.

An example for a resulting tree is shown in fig. 4.9.

The RRT was originally presented for kinodynamic planning. The search space was not only positions/angles but a general state that could include velocities. [5]

Algorithm 2 Basic RRT

Require: start configuration q_{start}, goal or goal region
  add q_{start} as root node to the tree
  while goal or goal region is not reached **do**
    $q_{random} \leftarrow$ get random configuration
    $q_{nearest} \leftarrow$ get nearest configuration of q_{random} in the tree
    $q_{new} \leftarrow q_{nearest} + \Delta q$ in direction of q_{random}
    check the edge from $q_{nearest}$ to q_{new} for collision
    if no collision on the edge **then**
      add q_{new} as a child of $q_{nearest}$ to the tree
    end if
  end while

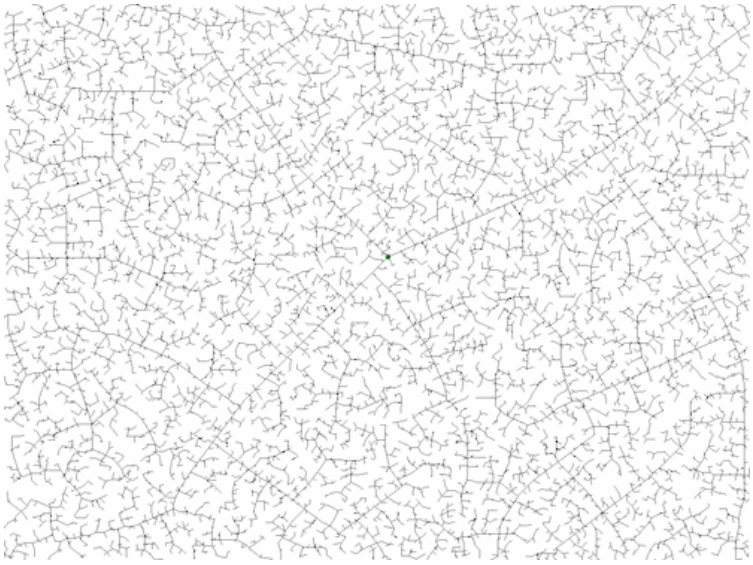


Figure 4.9: Example of PRM (extracted from Javed Hossain CC BY-SA 3.0, via Wikimedia Commons)

4.3 Continuous Description of Paths and Trajectories

This section deals with the continuous description of paths and trajectories. A path is the geometric location of the points along which the system moves. A trajectory is the time evolution of the state variables. It contains the geometric path together with the temporal information of how that path is traversed. The path required for a task results in part directly from the application. Examples are the application of adhesives along a line or welding processes.

Traditional robots are programmed in a manufacturer-specific language that specifies the parameters for the trajectories. These languages are very similar to the standard commands for CNC milling machines: G-Code. With G-Code or similar programming languages, the user tells the machine to move in a certain way. Often a fast movement, that is only approximately defined by a few viapoints, is used to approach the location where the tool is to do some work. Then, a precise point-to-point movement, a straight line, lets the tool approach the workpiece. In this second, step approximate paths are often not suitable because the robot is close to obstacles or the tool of the milling machine starts beeing in contact with the workpiece. In the following, we will to discuss approaches to these different needs.

Modern more autonomous robots may work with an environment map and plan their

paths using a discrete method first. Here, we want to show possible approaches how to describe the linear point-to-point movements that we have used for local planners in PRM and RRT. We also want to connect points from the discrete planner in a more continuous way.

Both paths and trajectories can be described in work and joint space. It may be necessary to describe the motion of the tool close to the workpiece in the workspace so that the tool does not collide unintentionally. When a human user specifies viapoint, it is easier to imagine the path between them in the workspace. Since the workspace requires inverse kinematics, singularities are a problem for the controller. Even if the robot is only close to a singularity, certain joints have to move very fast. Especially for more autonomous robots the joint space can be a good alternative. For redundant robots, only the joint angles define the configuration of the robot that is needed to calculate distances to the environment.

Limits

Before we can start planning a trajectory we need to know the limitations of the robot. In general joint angles and torques are limited. There are two main aspects that limit the torques: One is a maximum force at the teeth of the gears. The other is the current, which is proportional to the torque. High currents cause thermal damage to the motor windings. These two limits, together with the equation of motion, are difficult to use when planning trajectories with velocities and accelerations. Therefore, maximum values for joint velocities and joint accelerations are often estimated:

$$
\begin{aligned}
\boldsymbol{q}_{\min} \leq \boldsymbol{q} &\leq \boldsymbol{q}_{\max} \\
\boldsymbol{Q}_{\min} \leq \boldsymbol{Q}_u &\leq \boldsymbol{Q}_{\max} \\
\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} + \boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}}) &= \boldsymbol{Q}_u \\
&\downarrow \\
\boldsymbol{q}_{\min} \leq \boldsymbol{q} &\leq \boldsymbol{q}_{\max} \\
\dot{\boldsymbol{q}}_{\min} \leq \dot{\boldsymbol{q}} &\leq \dot{\boldsymbol{q}}_{\max} \\
\ddot{\boldsymbol{q}}_{\min} \leq \ddot{\boldsymbol{q}} &\leq \ddot{\boldsymbol{q}}_{\max}
\end{aligned}
$$

Continuity

Let us parameterize a path with a path parameter $s(t)$:

$$
\begin{aligned}
\boldsymbol{q} &= \boldsymbol{q}(s(t)) \\
\dot{\boldsymbol{q}} &= \boldsymbol{q}'(s)\, \dot{s}(t) \\
\ddot{\boldsymbol{q}} &= \boldsymbol{q}''(s)\, \dot{s}(t)^2 + \boldsymbol{q}'(s)\, \ddot{s}(t)
\end{aligned}
$$

Deriving $\boldsymbol{q}$ with respect to the time applying the chain rule, we see that we need $\boldsymbol{q}(s)$ and $s(t)$ to be of class C^2 if we want to have a continuous second time derivative. The second time derivative, the acceleration, is proportional to the current. So jumps in the accelerations are jumps in the current. Since current controllers are very fast, jumps in the acceleration are acceptable but lead to vibrations on the robot, because in reality it is not as stiff as we modeled it.

4.3.1 Point-To-Point in Joint Space

We start looking at paths and trajectories that connect two points in the joint space. Point-to-point (PTP) movements are normally understood as straight lines between the two points. We can write this path from start to goal as

$$\boldsymbol{q}(s) = \boldsymbol{q}_{\text{start}} + \left(\boldsymbol{q}_{\text{goal}} - \boldsymbol{q}_{\text{start}}\right) s$$

with $s \in [0, 1]$. In this section we look at different functions for s.

Trapezoidal Profiles

If we accept jumps in the acceleration one joint can accelerate with the highest possible acceleration untill it reaches the highest possible velocity. Then the joints moves with this maximum velocity and decelerates with the highest possible deceleration. These three phases lead to a trapezoidal velocity profile. The trajectory of one joint q_i is the trajectory of the pathparameter s scaled with Δq_i:

$$\begin{aligned}
\boldsymbol{q}(s) &= \boldsymbol{q}_{\text{start}} + \left(\boldsymbol{q}_{\text{end}} - \boldsymbol{q}_{\text{start}}\right) s \\
\boldsymbol{q}'(s) &= \boldsymbol{q}_{\text{end}} - \boldsymbol{q}_{\text{start}} = \Delta \boldsymbol{q} \\
\boldsymbol{q}''(s) &= 0 \\
\dot{\boldsymbol{q}} &= \boldsymbol{q}' \, \dot{s} = \Delta \boldsymbol{q} \, \dot{s} \\
\ddot{\boldsymbol{q}} &= \underbrace{\boldsymbol{q}'' \, \dot{s}^2}_{=\boldsymbol{0}} + \boldsymbol{q}' \, \ddot{s} = \Delta \boldsymbol{q} \, \ddot{s}
\end{aligned}$$

So the shape, that can be seen in fig. 4.10, is the same for all q_i and s. When applying the idea of trapezoidal velocity profiles to a robot with several degrees of freedom, always the first joint, that reaches its maximum, limits the system.

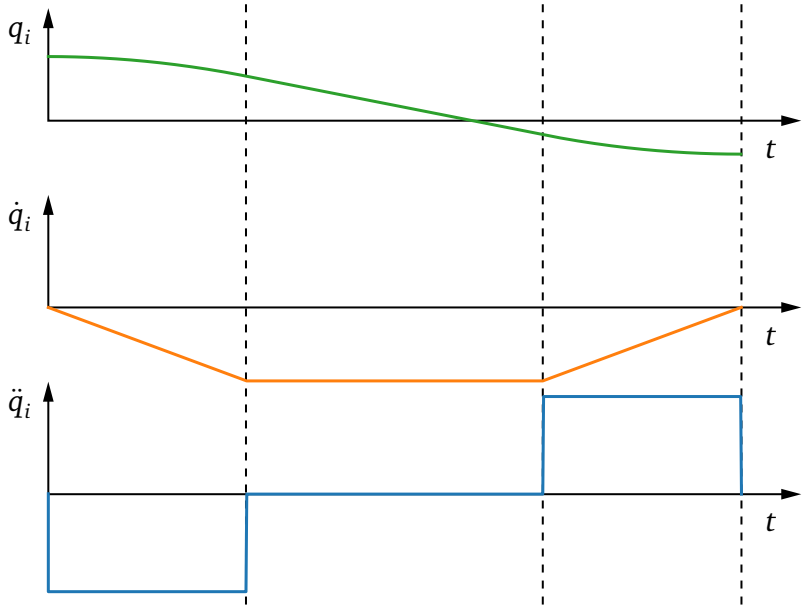


Figure 4.10: PTP Trajectory with Trapezoidal Velocity Profiles

The same idea can be applied one derivative higher starting with a maximum jerk. This leads to trapezoidal acceleration profiles.

Polynomial Profiles

An other possibility is a polynomial. A cubic polynomial

$$k_3 t^3 + k_2 t^2 + k_1 t + k_0$$

comes with four parameters. These parameters can be chose such that

$$s(0) = 0 \qquad s(T) = 1$$
$$\dot{s}(0) = 0 \qquad \dot{s}(T) = 0$$

Figure 4.11 shows the resulting PTP trajectory. With a quintic polynomial with six parameters also the acceleration at the beginning and the end can be set to zero.

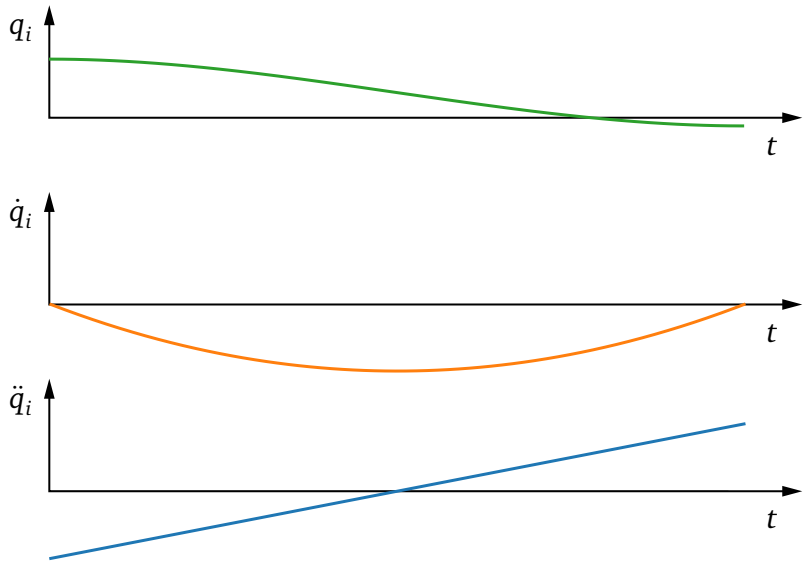


Figure 4.11: PTP Trajectory with a Cubic Polynomial Profil

4.3.2 Multiple Points in Joint Space

Now we want to connect mulitple points in the joint space. Here, we differentiate between

- Viapoints
  that control the shape of the path but are not necessarily part of the path and

- Thoughpoints
  that control the shape of the path and the path goes through them.

A first idea could be to use multiple PTP trajectories. This results in a stop and go motion as the velocity has to be zero at each start/goal point. The direction of the velocity before and after a start/goal point is different. As jumps in the velocity are a really bad assumption, it has to be zero at every start/goal point.

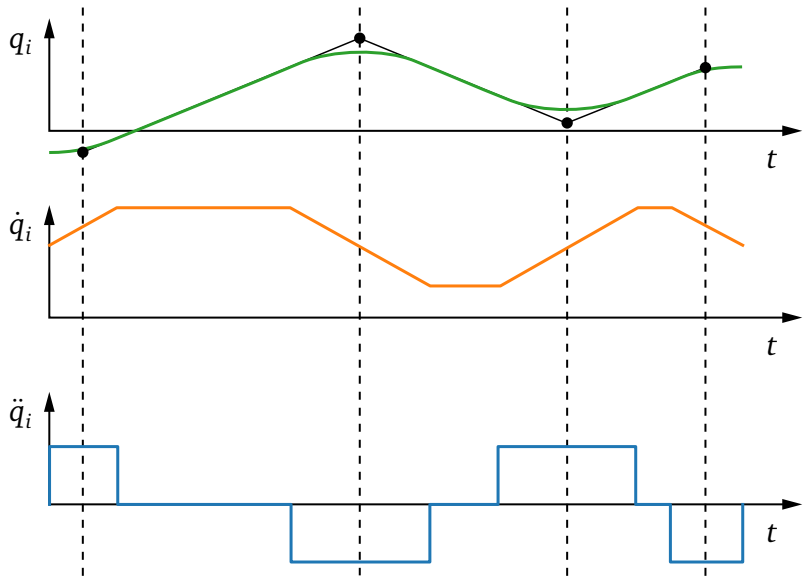


Figure 4.12: Trajectory with Parabolic Blends

Trapezoidal Profiles

Let's us first connect the points with a straight line. Travelling along this path without stopping at the points would lead to jumps in the velocity. Therefore, we allow the path to blend over between one straight connection line and the next one. We add such a blend at each point. Note that the points are generally not reached, they are Viapoints!

It is normal to use a constant velocity along the straight line and only accelerate during the blending. Using a constant acceleration for the blending leads to trapezoidal velocity profiles and parabolic position profiles – this method is often called Parabolic Blends. The constant acceleration can be the maximum possible acceleration as shown in fig. 4.12 There are also methods where the user can specify a radius around the viapoints. The blending is then only allowed within this radius.

As with the trapezoidal profiles for PTP trajectories one can also translate this method to jumps only in higher derivations like the jerk.

Polynomial Profiles

To connect multiple points with polynomials, we can use a similar approach as for PTP trajectories. We just don't force the velocity and acceleration to be zero at the through-points. This is called spline interpolation. The polynomials are calculated separately for each joint. They are **not** synchonised! With n polynomials, we can connect $n+1$ through-points. Using cubic polynomials we get $4n$ polynomial constants for each joint. These constants can be chosen to satisfy certain conditions. For example, we could ask for zero velocity at the start and the goal, and for continuous velocity and acceleration at the

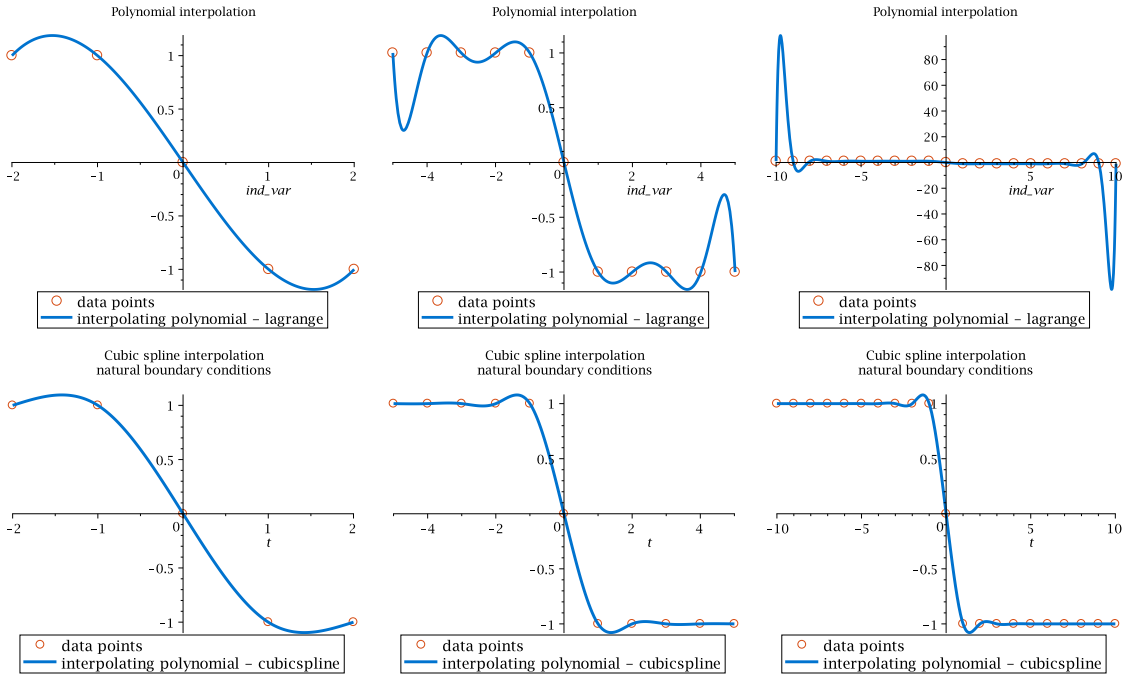


Figure 4.13: Higher Order Polynomial vs. Spline: Interpolation with one polynomial in the first row and with cubic splines in the second row

internal throughpoints, as this gives $4+4(n-1)$ conditions:

start

$$q_{i,S0}(t_0) = q_{i,\text{start}}$$
$$\dot{q}_{i,S0}(t_0) = 0$$

goal

$$q_{i,Sn}(t_n) = q_{i,\text{goal}}$$
$$\dot{q}_{i,Sn}(t_n) = 0$$

internal throughpoints j

$$q_{i,Sj}(t_j) = q_{i,Sj+1}(t_j) = q_{i,j}$$
$$\dot{q}_{i,Sj}(t_j) = \dot{q}_{i,Sj+1}(t_j)$$
$$\ddot{q}_{i,Sj}(t_j) = \ddot{q}_{i,Sj+1}(t_j)$$

There are several approaches to obtain trajectories with continuous accelerations. One is to use higher order polynomials for the spline. Another method is to realise the start and the goal using two points to get zero velocity and zero acceleration at both ends of the trajectory.

Why do we use multiple polynomials to interpolate the points, rather than one with a higher degree? Higher order polynomials tend have large oscillations as shown in fig. 4.13. Splines also lead to small overshoots but they are much smaller than with higher order polynomial interpolation.

4.4 Path and Trajectory Optimization

In the previous parts of this chapter, we have seen how waypoints can be found to link a starting state to an end state, typically for states in the workspace (W-space) or in

the configuration space (C-space), and how the waypoints can be linked by continuous functions to create a trajectory.

Although the trajectories found by such sampling-based planners have been obtained by looking for some optimum according to specific heuristics, they are often modified in a post-processing step because

- the optimization is based on a discrete set of possible states
- the heuristics used in the sampling-based approaches are usually approximate
- not all constraints are usually considered in the sampling-based solution
- if the planning was done in W-space, one needs to find a corresponding trajectory in C-space (inverse kinematics)

4.4.1 Simple Example: Inverse Kinematics

In order to start with a simple illustrative trajectory optimization example, imagine that a feasible trajectory has been obtained in W-space by sampling-based techniques. In order to command the motors of the robot such that the planned W-space trajectory is followed, we need to find the related trajectory in C-space such that, at velocity level, (see Chapter 2)

$$\boldsymbol{J}_w(\boldsymbol{q})\ \dot{\boldsymbol{q}} = \dot{\boldsymbol{w}}_d \tag{4.4.1}$$

If the robot has as many joints as the dimension of the W-space (no redundancy), then joint velocities can be found by

$$\dot{\boldsymbol{q}} = \boldsymbol{J}_w^{-1}(\boldsymbol{q})\ \dot{\boldsymbol{w}}_d \tag{4.4.2}$$

This was called the "Inverse Jacobi" method, which, in its discrete implementation, needed a drift control. We see that in the case of a non-redundant robot, there is nothing to optimize in the inverse kinematics.

If the robot is however redundant, the solution of eq. (4.4.1) is not unique and one can search for a specific solution that is in some sense optimum, namely the solution that minimizes the joints velocities (for instance in an attempt to reduce wear or friction losses). So the optimum $\dot{\boldsymbol{q}}^*$ is defined as the solution of

$$\begin{cases} \min\limits_{\dot{\boldsymbol{q}}}\ \dot{\boldsymbol{q}}^T \boldsymbol{W} \dot{\boldsymbol{q}} \\ \text{subject to} \quad \boldsymbol{J}_w(\boldsymbol{q})\ \dot{\boldsymbol{q}} - \dot{\boldsymbol{w}}_d = \boldsymbol{0} \end{cases} \tag{4.4.3}$$

This approach was called the "Resolved Motion Rate Control". This formulation is typical for an optimization problem under constraints. The solution of this problem was found in Chapter 2 using the technique of Lagrange multipliers.

Finding an optimum of a cost function under constraints is at the heart of optimization techniques as used in trajectory optimization for instance and is often referred as **Nonlinear Programming** (in the general case were the function to minimize and/or the constraints are nonlinear). In the special case where the cost function minimized in the process is quadratic and the constraints are linear, one talks about **Quadratic Programming**.

To discuss how the optimization can be expressed in a solvable form, and later how the optimal solution can be numerically found, we need to go a bit deeper in the underlying mathematical concepts.

4.4.2 Lagrange Multipliers

In order to understand the tools used to express and solve minimization problems arising in optimization and in optimal control (which is basically what optimal trajectory planning is), we need to do a bit of applied mathematics ... hopefully making it intuitive for engineers without going in rigorous proofs nor discussing all the details.

Let us call $\mathcal{C}$ the scalar cost function (also called "objective function") of the optimization variables $\boldsymbol{x}$ and assume the function to be convex[1]. $\boldsymbol{x} \in \mathbb{R}^n$ contains the n optimization variables. For the sake of the discussion, we consider the simple example

$$\mathcal{C}(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T \boldsymbol{W} \boldsymbol{x} + \boldsymbol{x}^T \boldsymbol{e} \tag{4.4.4}$$

where $\boldsymbol{W}$ is a symmetric positive definite matrix of weighting coefficients (indicating how much the elements in $\boldsymbol{x}$ contribute to the cost function) and $\boldsymbol{e}$ is a given vector. Note that here we use the notation $\boldsymbol{x}$ to denote whatever variable is involved in the cost function: it could represent workspace states (positions, orientations, velocities, accelerations),C-space degrees of freedom (or their velocities or accelerations) or control variables such as motor current or moments. For now, we consider those variables at a certain instant in time, not as time functions (see the discussion about time-local vs. time-global optimization later in section 4.4.4).

Minimizing $\mathcal{C}$ means that we search for the optimum $\boldsymbol{x}^*$ that is such that any variation $\delta\boldsymbol{x}$ around that solution does not further decrease the cost function. Mathematically,

$$\delta_x \mathcal{C} = \delta\boldsymbol{x}^T \left.\frac{d\mathcal{C}}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} = \delta\boldsymbol{x}^T(\boldsymbol{W}\boldsymbol{x}^* + \boldsymbol{e}) = 0 \qquad \forall\ \delta\boldsymbol{x} \tag{4.4.5}$$

where we used the notation

$$\frac{d\mathcal{C}}{d\boldsymbol{x}} \overset{\text{def}}{=} \begin{bmatrix} d\mathcal{C}/dx_1 \\ \vdots \\ d\mathcal{C}/dx_n \end{bmatrix}$$

As eq. (4.4.5) must hold for all $\delta\boldsymbol{x}$

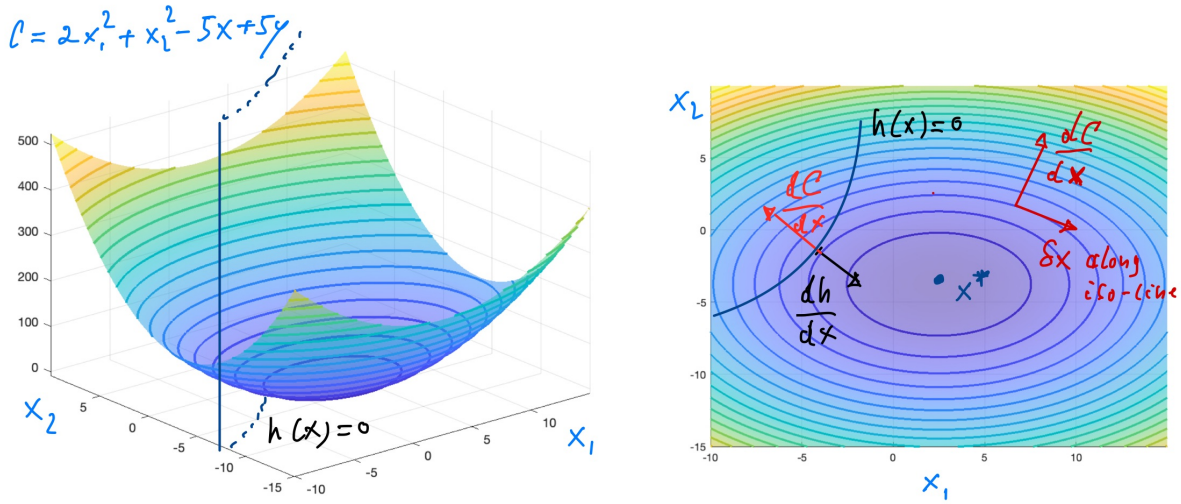
$$\left.\frac{d\mathcal{C}}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} = \boldsymbol{W}\boldsymbol{x}^* + \boldsymbol{e} = \boldsymbol{0} \tag{4.4.6}$$

resulting in the optimal solution $\boldsymbol{x}^* = -\boldsymbol{W}^{-1}\boldsymbol{e}$. A graphical interpretation for $n = 2$ can be seen in fig. 4.14. In that figure, the iso-lines representing the curves for which $\mathcal{C}$ is constant are also shown. Around a certain $\boldsymbol{x}$, a variation $\delta\boldsymbol{x}$ keeping $\mathcal{C}$ constant (i.e. along an iso-line) would satisfy

$$\frac{d\mathcal{C}}{d\boldsymbol{x}}^T \delta\boldsymbol{x} = 0 \qquad \text{on iso-cost lines}$$

indicating that moving along an iso-line is orthogonal to the so-called gradient of the cost function $\frac{d\mathcal{C}}{d\boldsymbol{x}}$. In other words, $\frac{d\mathcal{C}}{d\boldsymbol{x}}$ is the direction in which the cost function increases the fastest and is thus sometimes called the direction of “steepest ascent” ($-\frac{d\mathcal{C}}{d\boldsymbol{x}}$ is called the direction of “steepest descent”). At the minimum, eq. (4.4.6) indicates that the gradient is null and there is no direction to further decrease the function.

[1] When convex, a first-order optimality condition is sufficient to characterize a global minimum



(a) (b)

Figure 4.14: Minimization under an equality constraint

If the minimum for $\mathcal{C}$ should be found under the condition that the optimization variables $\boldsymbol{x}$ satisfy an equality $h(\boldsymbol{x}) = 0$, than the constrained problem reads

$$\begin{cases} \min\limits_{\boldsymbol{x}} \mathcal{C} & \forall\ \delta\boldsymbol{x} \text{ satisfying} \\ h(\boldsymbol{x}) = 0 \end{cases} \tag{4.4.7}$$

Considering the graphical interpretation in fig. 4.14, we see that the solution $\boldsymbol{x}^*$ of this problem can no longer be found by searching for the minimum of $\mathcal{C}$ alone, but rather searching for the minimum only in the restricted space defined by $h(\boldsymbol{x}) = 0$.

Assume an arbitrary $\boldsymbol{x}$ satisfying the constraint. Following the same reasoning as for the steepest descent of $\mathcal{C}$, let us consider an admissible variation $\delta\boldsymbol{x}$, namely a variation satisfying the constraint around $\boldsymbol{x}$ and thus leaving constant the constraint $h = 0$, we can write

$$\frac{dh}{d\boldsymbol{x}}^T \delta\boldsymbol{x} = 0 \qquad \text{for } \delta\boldsymbol{x} \text{ satisfying the constraint}$$

Hence, an admissible variation $\delta\boldsymbol{x}$ must be orthogonal to the gradient of the constraint $\frac{dh}{d\boldsymbol{x}}$. In other words, $\frac{dh}{d\boldsymbol{x}}$ indicates the direction in which the constraint "acts".

We observe in fig. 4.14 that the minimum of the constrained problem is reached when changing $\boldsymbol{x}$ on the manifold $h(\boldsymbol{x}) = 0$ does not change the cost function $\mathcal{C}$; in other words, when the cost function $\mathcal{C}$ can only be reduced if one varies $\boldsymbol{x}$ in the direction $\frac{dh}{dx}$ orthogonal to the constraint, so violating the constraint. The fact that, at the minimum of the constrained problem, the gradient of the cost function and of the constraint must be parallel can be expressed as a linear combination by

$$\left.\frac{d\mathcal{C}}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \left.\frac{dh}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \lambda^* = \boldsymbol{0} \tag{4.4.8}$$

The scalar λ is introduced since the gradient of $\mathcal{C}$ and of h are not necessarily of the same magnitude (not even always pointing in the same direction). This parameter λ is known as a **Lagrange multiplier** and is determined such that the constraint $h(\boldsymbol{x}) = 0$ is satisfied.

Mathematically, this minimality condition can be expressed by defining a modified cost

function, the so-called **Lagrangian**,

$$\mathcal{L}(\boldsymbol{x},\lambda) \stackrel{\text{def}}{=} \mathcal{C}(\boldsymbol{x}) + \lambda h(\boldsymbol{x}) \tag{4.4.9}$$

and stating that the optimal solution $(\boldsymbol{x}^*,\lambda^*)$ is the one that renders stationary the Lagrangian for any variation of $\boldsymbol{x}$ and λ:

$$\boxed{\delta_{\boldsymbol{x},\lambda}\ \mathcal{L}(\boldsymbol{x}^*,\lambda^*) = 0 \qquad \forall\ \delta\boldsymbol{x},\delta\lambda} \tag{4.4.10}$$

Indeed, setting to zero the variation of the Lagrangian with respect to $\boldsymbol{x}$ and λ yields:

$$\begin{cases} \delta\boldsymbol{x}^T \dfrac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = 0 & \forall\ \delta\boldsymbol{x} \\ \delta\lambda^T \dfrac{\partial \mathcal{L}}{\partial \lambda} = 0 & \forall\ \delta\lambda \end{cases} \quad \Leftrightarrow \quad \boxed{\begin{cases} \left.\dfrac{d\mathcal{C}}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \left.\dfrac{dh}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \lambda^* = \boldsymbol{0} \\ h(\boldsymbol{x}^*) = 0 \end{cases}} \tag{4.4.11}$$

The set of eq. (4.4.11) are defining the optimal solution of the constrained problem.

Example

In order to illustrate the concept of Lagrange multipliers and stationarity of the Lagrangian function, let us consider as cost function the potential of a spring in a plane, attached at the origin and having a zero natural length. When a force $\boldsymbol{F}$ is applied to the spring, the total potential of the system writes

$$\mathcal{C} = \frac{1}{2}k\|\boldsymbol{x}\|^2 - \boldsymbol{x}^T\boldsymbol{F} = \frac{1}{2}k(x_1^2+x_2^2) - (F_1x_1 + F_2x_2)$$

where k is the stiffness of the spring, $\|\boldsymbol{x}\|$ can be interpreted as the deformation of the spring. The unconstrained minimality condition yields

$$\frac{d\mathcal{C}}{d\boldsymbol{x}} = \begin{bmatrix} \frac{d\mathcal{C}}{dx_1} \\ \frac{d\mathcal{C}}{dx_2} \end{bmatrix} = \begin{bmatrix} kx_1^* - F_1 \\ kx_2^* - F_2 \end{bmatrix} = \boldsymbol{0}$$

which correspond to the equilibrium condition for the system shown in fig. 4.15.a.

If now, in addition, the end of the spring is constraint to move on a parabola

$$x_2 = (x_1-1)^2 + 2$$

then the minimum has to satisfy the equality constraint

$$h(\boldsymbol{x}) = (x_1-1)^2 - x_2 + 2 = 0$$

The Lagrangian of the problem writes

$$\mathcal{L}(\boldsymbol{x},\lambda) = \frac{1}{2}k(x_1^2+x_2^2) - (F_1x_1+F_2x_2) + \big[(x_1-1)^2 - x_2 + 2\big]\lambda$$

Writing that the Lagrangian is stationary for any variation in $\boldsymbol{x}$ and λ results in

$$\delta_{\boldsymbol{x}}\ \mathcal{L}(\boldsymbol{x},\lambda) \ \rightarrow\ \underbrace{\begin{bmatrix} kx_1^* - F_1 \\ kx_2^* - F_2 \end{bmatrix}}_{\frac{d\mathcal{C}}{d\boldsymbol{x}}} + \underbrace{\begin{bmatrix} 2(x_1^*-1) \\ -1 \end{bmatrix}}_{\frac{dh}{d\boldsymbol{x}}} \lambda^* = \boldsymbol{0} \qquad \text{(Euler-Lagrange equation)}$$

$$\delta_{\lambda}\ \mathcal{L}(\boldsymbol{x},\lambda) \ \rightarrow\ (x_1^*-1)^2 - x_2^* + 2 = 0 \qquad (h(\boldsymbol{x}^*) = 0)$$

This results in 3 equations for 3 unknowns that determine the constrained equilibrium position of the system, see fig. 4.15.b. It can be observed that for this problem, the Lagrange multiplier can be interpreted as the reaction force acting in the direction dh/dx in order to keep the spring on the parabola.

Although this example is not related to trajectory optimization, the problem would be mathematically identical to the minimization in the Resolved Motion Rate Control eq. (4.4.3) if we set $\boldsymbol{F} = 0$, take as

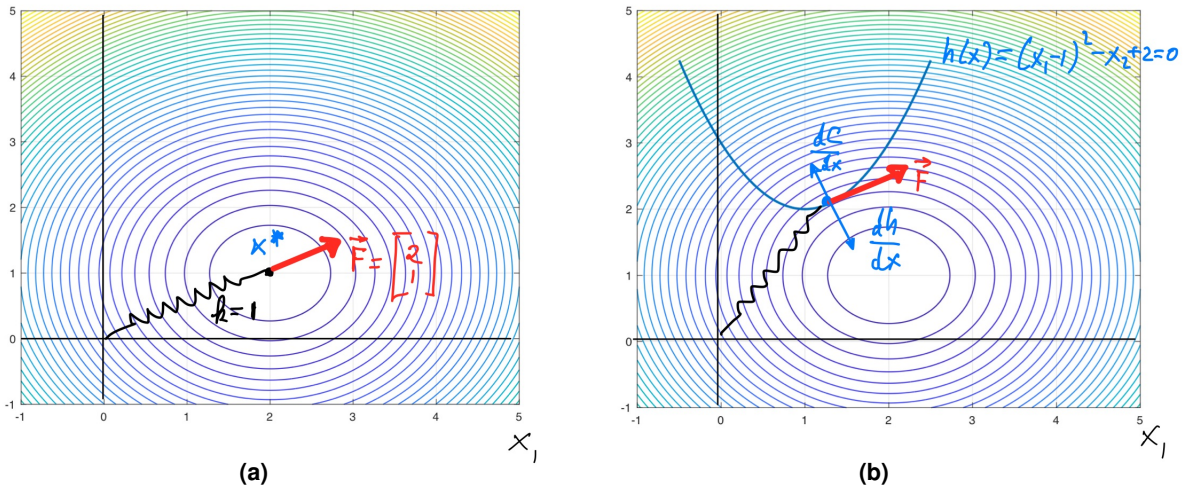


Figure 4.15: Minimization problem for a spring, with and without equality constraint

variable $[x_1, x_2]$ the joint velocities $[\dot{q}_1, \dot{q}_2]$, and consider the parabola as the constraint representing the kinematic relation $J_w(x_1, x_2) - \dot{w}_d = 0$ yielding the desired one dimensional workspace velocity $\dot{w}_d$.

In general, if the variables are submitted to $m < n$ equality conditions

$$\boldsymbol{h}(\boldsymbol{x}) = \begin{bmatrix} h_1(\boldsymbol{x}) \\ \vdots \\ h_m(\boldsymbol{x}) \end{bmatrix} = \boldsymbol{0} \qquad \boldsymbol{\lambda} = \begin{bmatrix} \lambda_1 \\ \vdots \\ \lambda_m \end{bmatrix}$$

the Lagrangian and the stationarity condition is expressed as

$$\boxed{\begin{array}{l} \mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) \stackrel{\text{def}}{=} C(\boldsymbol{x}) + \boldsymbol{h}(\boldsymbol{x})^T \boldsymbol{\lambda} \\ \\ \delta_{\boldsymbol{x},\boldsymbol{\lambda}}\ \mathcal{L}(\boldsymbol{x}^*, \boldsymbol{\lambda}^*) = 0 \qquad \forall\ \delta\boldsymbol{x}, \delta\boldsymbol{\lambda} \end{array}} \tag{4.4.12}$$

where a Lagrange multiplier λ_i is introduced for each equality condition h_i. Similarly to eq. (4.4.11), the minimality conditions are then obtained as

$$\left\{ \begin{array}{ll} \delta\boldsymbol{x}^T \dfrac{\partial \mathcal{L}}{\partial \boldsymbol{x}} = 0 & \forall\ \delta\boldsymbol{x} \\ \\ \delta\boldsymbol{\lambda}^T \dfrac{\partial \mathcal{L}}{\partial \boldsymbol{\lambda}} = 0 & \forall\ \delta\boldsymbol{\lambda} \end{array} \right. \quad \Leftrightarrow \quad \boxed{\left\{ \begin{array}{l} \left.\dfrac{dC}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \left.\dfrac{d\boldsymbol{h}^T}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \boldsymbol{\lambda}^* = \boldsymbol{0} \\ \\ \boldsymbol{h}(\boldsymbol{x}^*) = \boldsymbol{0} \end{array} \right.} \tag{4.4.13}$$

where we use the notation[2]

$$\frac{d\boldsymbol{h}^T}{d\boldsymbol{x}} = \begin{bmatrix} \frac{dh_1}{dx_1} & \frac{dh_2}{dx_1} & \cdots & \frac{dh_m}{dx_1} \\ \vdots & & & \\ \frac{dh_1}{dx_n} & \frac{dh_2}{dx_n} & \cdots & \frac{dh_m}{dx_n} \end{bmatrix}$$

[2]This notation means

$$\frac{d\boldsymbol{h}^T}{d\boldsymbol{x}} \stackrel{\text{def}}{=} \frac{d}{d\boldsymbol{x}} \boldsymbol{h}^T = \begin{bmatrix} \frac{d}{dx_1} \\ \vdots \\ \frac{d}{dx_n} \end{bmatrix} \begin{bmatrix} h_1 & h_2 & \cdots & h_m \end{bmatrix}$$

The interpretation of the Euler-Lagrange equation eq. (4.4.13)

$$\left.\frac{d\mathcal{C}}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \left.\frac{d\boldsymbol{h}^T}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \boldsymbol{\lambda}^* = \left.\frac{d\mathcal{C}}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \sum_{i=1}^{m} \left.\frac{dh_i}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \lambda_i^* = \boldsymbol{0}$$

is similar to what was discussed for the case of a single constraint (fig. 4.14): at the optimum solution, only modifications of $\boldsymbol{x}$ that violate the constraints can minimize $\mathcal{C}$ further.

4.4.3 Inequality conditions - KKT conditions

Let us now consider the more difficult but quite common case where the minimization problem is subject to an inequality constraint. Assuming that one only has one inequality constraint for $\boldsymbol{x}$, the minimization problem eq. (4.4.7) is modified and written in the general form

$$\begin{cases} \min\limits_{\boldsymbol{x}} \ \mathcal{C} & \forall\ \delta\boldsymbol{x} \\ \text{subject to} \ \ g(\boldsymbol{x}) \leq 0 \end{cases} \tag{4.4.14}$$

where $g(\boldsymbol{x})$ is a function formulated, by convention, such that it should always be negative or null. For the spring example seen later (where the end point of the spring is restricted in an area by an infinitely hard obstacle), $g(\boldsymbol{x})$ represents the penetration into the obstacle.

This case can also be treated by a Lagrange multiplier technique as for the equality conditions seen in the previous section, but now two different cases need to be distinguished:

- **constraint is inactive** – Either the minimum $\boldsymbol{x}^*$ of the cost function $\mathcal{C}$ lays in a region where $g(\boldsymbol{x}) < 0$, in which case the inequality condition has no effect on the optimum solution and can be disregarded (fig. 4.16.a). In other word, we must set the Lagrange multiplier λ to 0 in the Lagrangian eq. (4.4.9) and in the Lagrange-Euler eq. (4.4.8). We will say that the constraint in *inactive*.
- **constraint is active** – Or the minimum of $\mathcal{C}$ is in the region where the inequality is violated (fig. 4.16.b), in which case the minimum of $\mathcal{C}$ alone cannot be the optimal solution. One then needs to consider the Lagrangian eq. (4.4.9) as modified cost function and compute the minimum with the Euler-Lagrange eq. (4.4.8) where λ is computed such that the optimum is found on the border of the admissible region, namely $g(\boldsymbol{x}) = 0$.

Let us call μ the Lagrange multiplier related to an inequality constraint[3]: from the discussion above, either $\mu = 0$ and $g(\boldsymbol{x}) \neq 0$ (constraint inactive) or $g(\boldsymbol{x}) = 0$ and $\mu \neq 0$ (constraint active). This can be summarized by

$$\mu^*\ g(\boldsymbol{x}^*) = 0 \tag{4.4.15}$$

and is called the complementarity condition. The minimality conditions could thus be written similar to eq. (4.4.11).

[3] Lagrange multipliers for an inequality behave a bit different from the λ introduced for equalities and are given a special notation to avoid confusion, although they basically have similar roles in the minimization process.

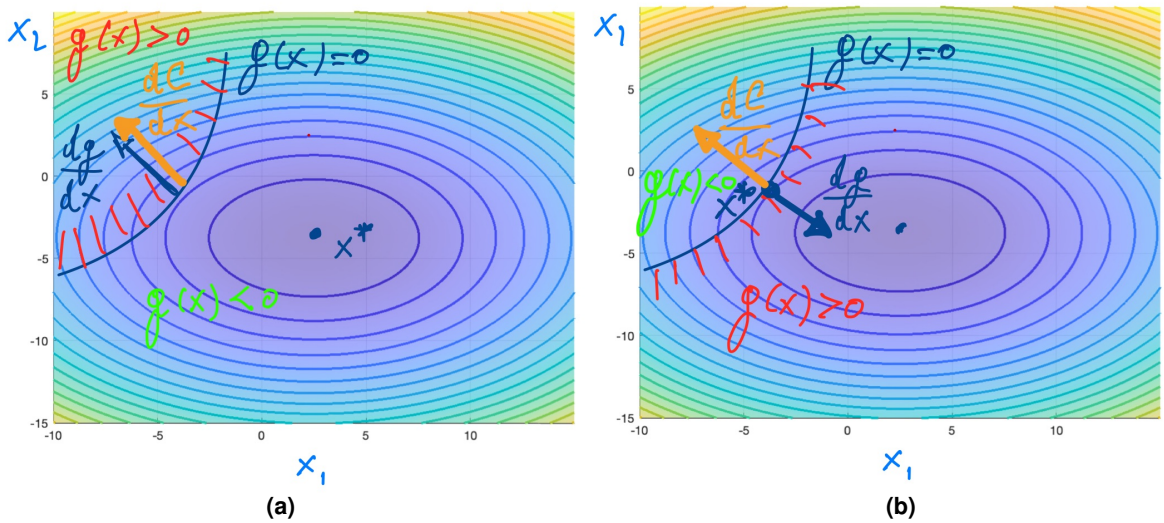


Figure 4.16: Minimization under an inequality constraint

$$\begin{cases} \left.\dfrac{dC}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \left.\dfrac{dg}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \mu^* = \boldsymbol{0} \\ g(\boldsymbol{x}^*) \leq 0 \\ \mu^*\, g(\boldsymbol{x}^*) = 0 \end{cases} \tag{4.4.16}$$

These conditions are however not sufficient since it would allow having a solution on the boundary $g(\boldsymbol{x}^*) = 0$, $\mu \neq 0$ even though the minimum of the cost function is in the admissible region, as if the constraint would be an equality (see fig. 4.16). So one needs a last condition that indicates that, if one is on the boundary ($\mu \neq 0$), a change of the optimum $\boldsymbol{x}^*$ to further minimize C would bring the solution in the non-admissible region of the constraint.

For this, let us consider fig. 4.16 and remember that the gradient $\frac{dg}{d\boldsymbol{x}}$ and $\frac{dC}{d\boldsymbol{x}}$ point in the direction of steepest ascent of g and C, respectively (see previous section). Hence, $\frac{dg}{d\boldsymbol{x}}$ points from the boundary $g = 0$ into the direction where the constraint would be violated. In the case of fig. 4.16.a, the direction $-\frac{dC}{d\boldsymbol{x}}$ from the boundary to a smaller C points into the admissible region, showing that the solution can be further improved and is thus not optimum. So in that case, if one would compute the optimum by sticking on the boundary $g = 0$, one would solve the Euler-Lagrange equation (previous section):

$$\frac{dC}{d\boldsymbol{x}} + \mu \frac{dg}{d\boldsymbol{x}} = 0$$

and find a negative μ since $\frac{dg}{d\boldsymbol{x}}$ and $\frac{dC}{d\boldsymbol{x}}$ point in the same direction. Hence, if one finds a solution with a positive Lagrange multiplier, one is not in the optimum, although one satisfies eq. (4.4.16).

For the case of fig. 4.16.b, being on the boundary will yield the optimal solution since a further decrease of the cost function would mean entering the forbidden region $g > 0$. Here, solving the Euler-Lagrange equation for the equality case $g = 0$

$$\frac{dC}{d\boldsymbol{x}} + \mu \frac{dg}{d\boldsymbol{x}} = 0$$

would now yield a positive μ since $\frac{dg}{d\boldsymbol{x}}$ and $\frac{dC}{d\boldsymbol{x}}$ point in opposite direction.

In summary, the optimum solution in case of an inequality must satisfy the conditions

$$\boxed{\begin{cases} \left.\dfrac{dC}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} + \left.\dfrac{dg}{d\boldsymbol{x}}\right|_{\boldsymbol{x}^*} \mu^* = \boldsymbol{0} \\ g(\boldsymbol{x}^*) \leq 0 \\ \mu^* \; g(\boldsymbol{x}^*) = 0 \\ \mu^* \geq 0 \end{cases}} \tag{4.4.17}$$

These conditions are known as the **Karush-Kuhn-Tucker** (KKT) conditions[4]

Example

Let us consider again the simple example of a spring, but now in the presence of an inequality condition. In fig. 4.17.a, we assume that there is a boundary line defined by

$$x_2 = -0.4x_1 + 4$$

so defining the inequality condition

$$g(x_1, x_2) = 0.4x_1 + x_2 - 4 \leq 0$$

is satisfied for the region under or on the line. Reminding (see previous example) that

$$\mathcal{C} = \frac{1}{2}k\|\boldsymbol{x}\|^2 - \boldsymbol{x}^T\boldsymbol{F} = \frac{1}{2}k(x_1^2 + x_2^2) - (F_1x_1 + F_2x_2) = 0.5(x_1^2 + x_2^2) - (2x_1 + x_2)$$

where we took $k = 1$ and $\boldsymbol{F} = [2, 1]$. At the beginning of a numerical solution procedure, we do not know in which area of the solution space the optimum is located. Therefore, let us first assume that the solution satisfies $g = 0$ and find the solution for that assumption: the optimality would then write

$$\underbrace{\begin{bmatrix} x_1 - F_1 \\ x_2 - F_2 \end{bmatrix}}_{\frac{dC}{d\boldsymbol{x}}} + \underbrace{\begin{bmatrix} 0.4 \\ 1 \end{bmatrix}}_{\frac{dg}{d\boldsymbol{x}}} \mu = \boldsymbol{0} \qquad \text{(Euler-Lagrange equation)}$$

$$x_2 + 0.4x_1 - 4 = 0 \qquad (g(\boldsymbol{x}) = 0)$$

These three equations can be solved to yield

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 2.8 \\ 2.9 \end{bmatrix} \qquad \mu = -1.9$$

We thus see that the Lagrange multiplier ensuring the spring to be on the boundary is negative so that the solution on the boundary is not the optimum. Mechanically speaking, observing from the Euler-Lagrange equation above that the reaction force applied on the spring is

$$-\frac{dg}{d\boldsymbol{x}}\mu = \begin{bmatrix} 0.4 \\ 1 \end{bmatrix} 1.9$$

which indicates that the obstacle would need to pull on the spring, which is not possible. So the solution is in the region where $g < 0$ and where the constraint is not active. The solution is thus the unconstrained solution computed in the previous example.

In that case, we redefine g as being the negative of above, namely

$$g(x_1, x_2) = -0.4x_1 - x_2 + 4 \leq 0$$

the inequality condition now represents an obstacle forcing the optimum to be above or on the line $g = 0$ (fig. 4.17.b). The analysis is very similar to above: if one starts by assuming that the solution is on the boundary, we find the same set of equations as before, except that now $dg/d\boldsymbol{x}$ changes sign and thus the

[4] Sometimes the conditions eq. (4.4.13) in the absence of inequality constraints are also called KKT conditions.

solution gives a $\mu^* = 1.9$ which is positive. Hence the solution on the boundary is the optimum. If one would have started by assuming that the inequality constraint is non-active, one would have found the minimum of $\mathcal{C}$ and observed that it violates the constraint. One would then have reactivated the constraint and find the correct optimum.

It is interesting to see that, with inequalities, the solution procedure usually has to iterate in order to find if the constraints are active or not.

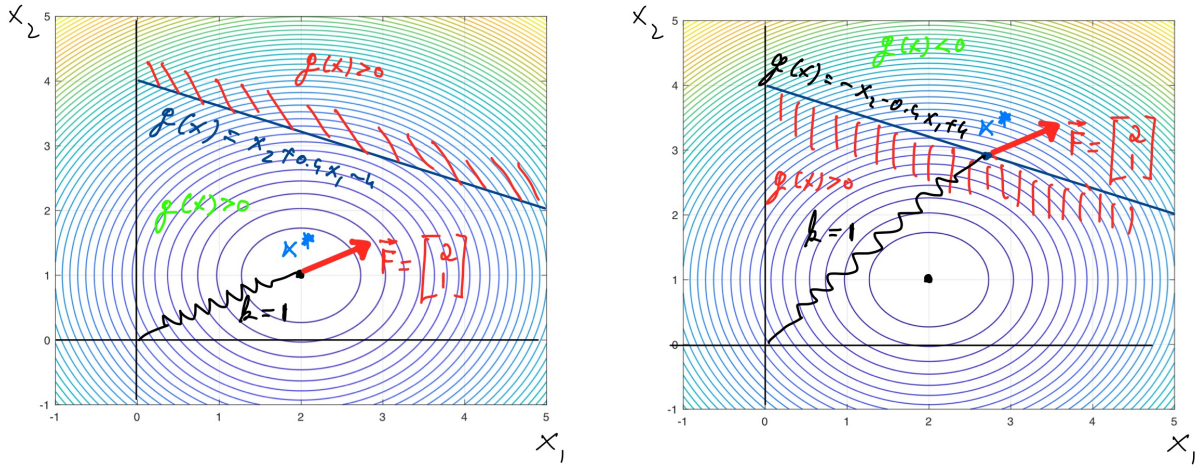


Figure 4.17: Minimization problem for a spring, with an inequality constraint

---

Obviously, if the minimization is subject to several equality and inequality conditions, one needs to introduce a Lagrange multiplier for each constraint.

Remark Since introducing constraints (equalities or inequalities) changes the nature of the minimization problem, one can account for the constraints in a weak sense by including so-called penalization contributions in the cost function, for instance

$$\mathcal{C}_{pen} = \mathcal{C} + \sum_i p_{h,i}\big(h_i(\boldsymbol{x})\big)^2 + \sum_j p_{g,j}[g_j(\boldsymbol{x})]_+^2 \tag{4.4.18}$$

where $p_{h,i}, p_{g,j}$ are very high penalty scalars and $[\ ]_+$ is non-zero only if the argument is positive. The optimum solution is then found by a classical minimization of a cost function, but the constraints are well satisfied only if the penalties are high enough, which can cause numerical problems.

4.4.4 Time-local vs. Time-Global Optimization

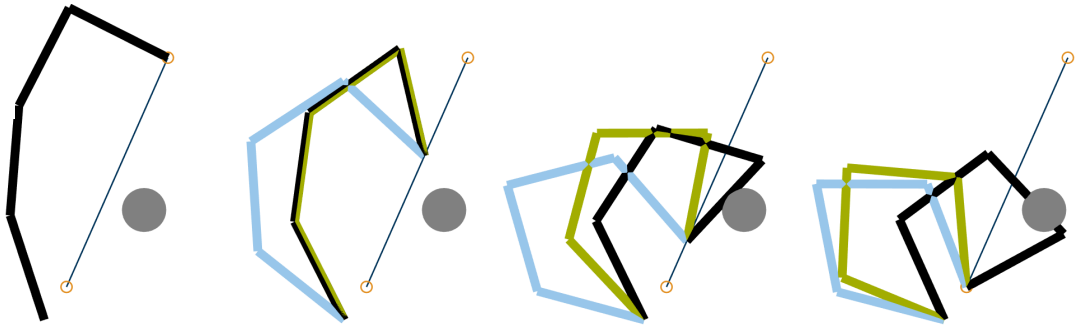
As seen in the previous section, the Resolved Motion Rate Control strategy eq. (4.4.3) to find joint velocities of a redundant robot for a desired workspace velocity can be seen as an optimization problem with equality constraints and quadratic cost function $\mathcal{C} = \dot{\boldsymbol{q}}^T \boldsymbol{W} \dot{\boldsymbol{q}}$.

One however has to note that the solution of the Resolved Motion Rate Control eq. (4.4.3) only satisfies an optimality condition for a certain moment in time, but does not optimize the cost over the full trajectory since the solution only considers the instantaneous information, not knowing what motion is required later. For that reason, we say that the inverse kinematic computed by an approach like eq. (4.4.3) (so Resolved Motion Rate Control or Automatic Supervisory Control) are **time-local optimization** approaches.

A **time-global optimization** would search for a solution that minimizes a cost over the full trajectory. For instance for the inverse kinematics problem, one could write the optimization problem as

$$\begin{cases} \min\limits_{\dot{q}} \ \Phi \quad \text{where } \Phi\big(\dot{q}(t)\big) = \int_{t_0}^{t_e} \underbrace{\dot{q}(t)^T \boldsymbol{W} \dot{q}(t)}_{\mathcal{C}(t)} \, dt \\ h(\dot{q}) = \boldsymbol{J}_w(q)\dot{q} - \dot{w}_d = \boldsymbol{0} \qquad \forall t \end{cases} \tag{4.4.19}$$

where t_e is the end time of the trajectory. The difference is now that the optimal joint velocities are not optimized separately for each instance, but such that the cost over the entire trajectory is optimum. That means that one could obtain joint velocities that are not optimal at a particular instance, but that allow later for lower joint velocities and thereby minimize the cost over the entire trajectory. As illustration, fig. 4.18 shows three snapshots of a redundant robot for which the inverse kinematic was computed with the Resolved Rate Motion approach (which is time-local and does not account for obstacle avoidance), the Automatic Supervisory Control approach (also time-local, but allowing to include the obstacle avoidance in the null space, see Chapter 2) and a time-global optimization. For all cases, the same starting state and workspace trajectory were taken, but clearly the time-global approach leads to overall smoother joint motions since the joint velocities are time-globally planned.



(a) $t = 0\,s$. (b) $t = 1.75\,s$. (c) $t = 2.55\,s$. (d) $t = 3.75\,s$.

Figure 4.18: Inverse kinematic obtained with time-local optimization strategies (RMRC in black, ASC in green), and with a time-global optimization (in blue). [3, 10]

In the time-global optimization eq. (4.4.19), we see that we are not only looking for a single solution, but actually for an optimal solution function $\dot{q}(t)$ over time. The cost $\mathcal{C}(\dot{q}(t))$ is actually a "function of a function", hence a so-called **functional**. Since the optimization problem is searched over a full trajectory, one typically speaks of **optimal control**.[5,6]

[5]Note that in English, the word "control" encompasses both the fact of planning the action (like in a feedforward control) as well as the monitoring and correction of the action as in a feedback control. In German, one would generally talk about "Planung/Steuerung" and "Regelung".

[6]If the motion is only optimized for a certain horizon, changing the horizon as the time passes, one speaks of Model Predictive Control.

Classically, the optimal control problem is written as

$$\begin{cases} \min\limits_{\boldsymbol{x},\boldsymbol{u}} \ \Phi \quad \text{where } \Phi = \displaystyle\int_{t_0}^{t_e} \mathcal{C}\big(\boldsymbol{x}(t),\boldsymbol{u}(t)\big)\, dt \\ \text{subject to} \\ \dot{\boldsymbol{x}} = \boldsymbol{a}(\boldsymbol{x},\boldsymbol{u}) \qquad \forall t \\ \boldsymbol{x}(t=t_0) = \boldsymbol{x}_0 \\ \boldsymbol{x}(t=t_e) = \boldsymbol{x}_e \end{cases} \tag{4.4.20}$$

This set of equations can be interpreted as follows:

One wants to minimize a cost over the full trajectory that is a function of the states $\boldsymbol{x}$ (in robotics typically positions, angles and their velocities) and also of the control input $\boldsymbol{u}$ (in robotics typically forces, moments, currents). Since the state trajectories are linked to the control input typically by a physical law generally of first order, the **state equation** (in robotics, the dynamic equations of the system), they cannot be varied independently. In addition, the initial states are given by the initial conditions of the state equations $\boldsymbol{x}_0$ and we require that the system should reach the final state $\boldsymbol{x}_e$, given at time t_e. Here, we assume the end time t_e of the trajectory is given.

4.4.5 Examples of Trajectory Optimization Strategies

In practice, the optimal control to be solved is more complicated than the simple form eq. (4.4.20). In here, we want to give some examples of optimal control encountered in the field of robotics.

Inverse Kinematics with Null-Space Control

The optimal control problem for the inverse kinematics eq. (4.4.19) looked for a solution simply minimizing a weighted norm of the joint velocities and we discussed that such a time-global optimum can better use the redundancy of the robot to minimize the velocities over the full trajectory. In the form eq. (4.4.19), the optimization variables were $\boldsymbol{x} = \dot{\boldsymbol{q}}$ and the constraints were not dynamic in the sense that we had simply $\boldsymbol{h}(\boldsymbol{x}) = \boldsymbol{0}$.

An alternative manner to exploit the redundancy of the robot would be to explicitly split the C-space solution into the part in the image of $\boldsymbol{J}_w$ which is determined by the given W-space, and another part in the nullspace that can be freely chosen over time (since it does not change the W-space) to optimize a certain cost function for the joint variables. The strategy would be described by the optimal control problem as [8, 10]

$$\begin{cases} \min\limits_{\boldsymbol{q},\boldsymbol{u}} \ \Phi \quad \text{where } \Phi = \displaystyle\int_{t_0}^{t_e} \mathcal{C}\big(\boldsymbol{q}(t),\boldsymbol{u}(t)\big)\, dt \\ \dot{\boldsymbol{q}} = \boldsymbol{J}_w^{\#}(\boldsymbol{q})\dot{\boldsymbol{w}}_d + \boldsymbol{N}_w\boldsymbol{u} \\ \boldsymbol{q}(t_0) = \boldsymbol{q}_0 \\ |\boldsymbol{q}(t)| \le \boldsymbol{q}_{max} \ ; \ |\dot{\boldsymbol{q}}(t)| \le \dot{\boldsymbol{q}}_{max} \end{cases} \tag{4.4.21}$$

where $\boldsymbol{J}_w^{\#}$ is a pseudo-inverse of the Jacobi matrix of the workspace and $\boldsymbol{N}_w = (I - \boldsymbol{J}_w^{\#}\boldsymbol{J}_w)$ is the kinematic nullspace (see Chapter 2). The control variable $\boldsymbol{u}$ is here a subspace of the joint velocities that can be used to minimize the cost function over the trajectory

while ensuring that the desired W-space trajectory is followed. Here, we decided not to specify the end configuration of the joint since the goal is to follow the desired W-space, which is guaranteed by the constraint. The cost function Φ is formulated in terms of joint degrees of freedom, so that this optimal control problem is of the general form eq. (4.4.20). We however added symmetric limit constraints (inequalities) for the joints positions and velocities.

As an example of cost function, one could define

$$\mathcal{C}(t) = c_1 \dot{\boldsymbol{q}}^T \dot{\boldsymbol{q}} + c_2 \boldsymbol{u}^T \boldsymbol{u} + \mathcal{C}_{col}(\boldsymbol{q})$$

where c_1, c_2 are coefficients that determine the weight between the different cost components and $\mathcal{C}_{col}$ is a penalization term that grows very fast when the robot approaches a configuration of collision with the environment or with itself. Note that the limit conditions in eq. (4.4.21) could also be introduced as an additional penalization term in the cost function.

Trajectory Optimization Accounting for the Dynamics of the Robot

In the previous example, the physical control variable of the motion (the moments in the motors) are not part of the optimization problem. They could be accounted for by considering the acceleration $\ddot{\boldsymbol{q}}$ in the C-space as being an image of the motor moments, and include them in the kinematic optimization problem like eq. (4.4.21). However, an exact representation of the motor moments can only be obtained when relating them to the motion by the dynamic equations of the robot. So a typical optimal control problem of a robot writes

$$\begin{cases} \min\limits_{\boldsymbol{x},\boldsymbol{u}} \ \Phi \quad \text{where } \Phi = \displaystyle\int_{t_0}^{t_e} \mathcal{C}\big(\boldsymbol{x},\boldsymbol{u}\big)\, dt \\ \dot{\boldsymbol{x}} = \boldsymbol{a}(\boldsymbol{x},\boldsymbol{u}) \\ \boldsymbol{x}(t_0) = \boldsymbol{x}_0 \ ; \ \boldsymbol{x}(t_e) = \boldsymbol{x}_e \\ |\boldsymbol{x}(t)| \leq \boldsymbol{x}_{max} \ ; \ |\boldsymbol{u}(t)| \leq \boldsymbol{u}_{max} \end{cases} \tag{4.4.22}$$

where $\boldsymbol{x}$ are the states of the joints (position and velocities) and $\boldsymbol{u}$ the moments of the motors (or of the joints). Note that if the end-time of the trajectory is not imposed, it can also be taken as a variale for the optimization: adding then for instance $(t_e - t_0)^2$ to Φ in eq. (4.4.22), the minimization would also try to minimize the duration of the motion. Also, one can use as terminal condition (i.e. at time t_e) that the states $\boldsymbol{x}(t_e)$ yield a given terminal W-space configuration. Another case would be when the entire W-space trajectory is given: one could then impose that $\boldsymbol{w}(\boldsymbol{q}) = \boldsymbol{w}_d$ using the (kinematics of the robot) or, at velocity level, that $\boldsymbol{J}_w \dot{\boldsymbol{q}} = \dot{\boldsymbol{w}}_d$. Since $\dot{\boldsymbol{q}}$ is included in the state space variables $\boldsymbol{x}$, this W-space constraint would yield an equality constraint for $\boldsymbol{x}$. Other equality or inequality constraints can be added to the problem either explicitly, or as a penalization term in the cost function as already explained earlier.

Minimization problems involving the dynamics of the robot are usually more costly to solve than optimal control problems that only take into account the kinematics of the system (for instance eq. (4.4.21)) since the dynamic equation (see Chapter 3) is more costly to build and to solve. In order to understand the costs of using the dynamic equation of the robot, let us remind from Chapter 3, that the projected Newton-Euler equations of a robot typically write like[7]

$$\boldsymbol{M}(\boldsymbol{q})\ddot{\boldsymbol{q}} = -\boldsymbol{h}(\boldsymbol{q},\dot{\boldsymbol{q}}) + \boldsymbol{u} \tag{4.4.23}$$

[7]Be aware that the notation $\boldsymbol{h}$ here is not related to the notation for the equality constraints introduced

which can be put in the state-space form

$$\begin{bmatrix} \boldsymbol{M}(\boldsymbol{q}) & \boldsymbol{0} \\ \boldsymbol{0} & \boldsymbol{I} \end{bmatrix} \frac{d}{dt} \begin{bmatrix} \dot{\boldsymbol{q}} \\ \boldsymbol{q} \end{bmatrix} = \begin{bmatrix} -\boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \\ \dot{\boldsymbol{q}} \end{bmatrix} + \begin{bmatrix} \boldsymbol{u} \\ \boldsymbol{0} \end{bmatrix} \qquad \longleftrightarrow \qquad \dot{\boldsymbol{x}} = \boldsymbol{a}(\boldsymbol{x}, \boldsymbol{u}) \tag{4.4.24}$$

We have seen in Chapter 3 that the mass matrix and the forces can be computed efficiently for robots with a tree-topology using recursive kinematics and recursive computation of the joint generalized forces $\boldsymbol{u}$. For the discussion of the solution techniques in the next section, it is important to remember that the notation $\dot{\boldsymbol{x}} = \boldsymbol{a}(\boldsymbol{x}, \boldsymbol{u})$ corresponds to a full dynamic problem: it can be used to compute $\boldsymbol{x}$ over time for a given joint control history $\boldsymbol{u}(t)$ (forward or direct dynamics) or compute the required control for a given states trajectory (inverse dynamics). Clearly, the direct dynamics is expensive since it involves solving a nonlinear problem where $\boldsymbol{x}_i$ depends on all inputs $\boldsymbol{u}_r$ that where applied before t_i. The inverse dynamics however is cheaper since $\boldsymbol{u}_i = \boldsymbol{M}(\boldsymbol{q}_i)\ddot{\boldsymbol{q}}_i + \boldsymbol{h}(\boldsymbol{q}_i, \dot{\boldsymbol{q}}_i)$ for any given time t_i.

4.4.6 Direct Optimization

There are many variants of optimal control problems for robots, depending on the purpose of the robot type, its number of joints and the limitations that need to be imposed. In the previous section, two examples were given. It is often desirable that the solution method for the optimal control problem allows for a real-time implementation. In this section, we shortly give an idea of what is involved when solving optimal problems numerically.

In the direct solution approach, the optimal problem is solved directly in its constrained minimum form like in eq. (4.4.21) or eq. (4.4.22). Later, in the next section, we will see that one can also first transform the problem in a set of differential equations (similar to the Euler-Lagrange equation discuss earlier) and then solve those.

To illustrate the steps in the numerical solution, let us consider the optimal problem eq. (4.4.22).

Discretization in time The first step to numerically solve the optimal control problem is to replace the continuous time functions of the optimization variables by a discretized representation. The trajectory of the joint variables $\boldsymbol{x}(t)$ in state space and the control input $\boldsymbol{u}(t)$ are in fact continuous time functions, but for the solution process we will consider them only at discrete times t_i:

$$\boldsymbol{x}(t) \longrightarrow \boldsymbol{x}_i = \boldsymbol{x}(t_i) \qquad i = 0, \dots n_t \tag{4.4.25}$$

$$\boldsymbol{u}(t) \longrightarrow \boldsymbol{u}_i = \boldsymbol{u}(t_i) \tag{4.4.26}$$

where $n_t + 1$ is the number of discretization points considered over the trajectory. One typically assumes that the time-steps $t_{i+1} - t_i$ are constant and equal to $\Delta t = (t_e - t_0)/n_t$. Since the discrete unknowns for $\boldsymbol{x}_i$ and $\boldsymbol{u}_i$ are taken at the same discrete time instance (so-called "collocation approach"), the integral over the trajectory needed to compute Φ can be approximated by a simple trapezoidal rule:

$$\Phi = \int_{t_0}^{t_e} \mathcal{C}\big(\boldsymbol{x}, \boldsymbol{u}\big)\, dt \simeq \sum_{i=0}^{n_t - 1} \frac{\Delta t}{2} \big(\mathcal{C}(\boldsymbol{x}_i, \boldsymbol{u}_i) + \mathcal{C}(\boldsymbol{x}_{i+1}, \boldsymbol{u}_{i+1})\big) \tag{4.4.27}$$

in this chapter, but represents the velocity-dependent inertia terms and non-control forces such as friction or gravity.

For the time discretization of the dynamic constraint in eq. (4.4.22), we assume that the relation between $\boldsymbol{x}_i$ and its time-derivatives $\dot{\boldsymbol{x}}_i$ is given for instance by the trapezoidal rule

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \frac{\dot{\boldsymbol{x}}_i + \dot{\boldsymbol{x}}_{i+1}}{2}\Delta t \tag{4.4.28}$$

where $\dot{\boldsymbol{x}}_i = \boldsymbol{a}(\boldsymbol{x}_i, \boldsymbol{u}_i)$ according to the dynamic constraint. So the dynamic constraint can be discretized as

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \frac{\boldsymbol{a}(\boldsymbol{x}_i, \boldsymbol{u}_i) + \boldsymbol{a}(\boldsymbol{x}_{i+1}, \boldsymbol{u}_{i+1})}{2}\Delta t \tag{4.4.29}$$

To summarize, the optimal control problem eq. (4.4.22) can be approximated by the discrete Nonlinear Programming (NLP) problem

$$\begin{aligned}
&\min_{\boldsymbol{x},\boldsymbol{u}} \quad \Phi \simeq \sum_{i=0}^{n_t-1} \frac{\Delta t}{2}\big(\mathcal{C}(\boldsymbol{x}_i, \boldsymbol{u}_i) + \mathcal{C}(\boldsymbol{x}_{i+1}, \boldsymbol{u}_{i+1})\big) \\
&\text{subject to} \\
&\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \frac{\boldsymbol{a}(\boldsymbol{x}_i, \boldsymbol{u}_i) + \boldsymbol{a}(\boldsymbol{x}_{i+1}, \boldsymbol{u}_{i+1})}{2}\Delta t \\
&\boldsymbol{x}_{i=0} = \boldsymbol{x}_0 \;;\; \boldsymbol{x}_{i=n_t} = \boldsymbol{x}_e \\
&|\boldsymbol{x}_i| \le \boldsymbol{x}_{max} \;;\; |\boldsymbol{u}_i| \le \boldsymbol{u}_{max} \qquad \forall i = 1, \dots n_t
\end{aligned} \tag{4.4.30}$$

The problem is now an algebraic Nonlinear Programming problem where the integral in the minimization function and the time-derivatives have vanished, at the cost of having now $3n \times n_t$ optimization variables, n being the number of degrees of freedom of the robot.[8] The NLP eq. (4.4.30) includes typically nonlinear functions of the optimization variables in the discretized cost function Φ and in the equality and inequality constraints. A numerical approach to solve this will be discussed next.

Remarks:

- For other optimal control problems, the discretization is done in a similar manner. Other discretization schemes can also be used. In particular, one could assume that the discrete variables $\boldsymbol{x}_i$ and $\boldsymbol{u}_i$ are the nodes of a spline representation of the trajectory (see previous sections). In that case, the time derivatives $\dot{\boldsymbol{x}}_i$ needed to discretize the dynamic constraint would be obtained from the spline expression directly and not from the trapezoidal rule used here above. Also, if one uses a spline for the representation of the trajectory, the integration of the cost function could be done analytically.

- For the discretization explained above, the trajectory given to the robot for execution is typically linearly interpolated in time between the discrete time instance.

- The discretized minimization problem eq. (4.4.30) can also be modified such, that only the control variables $\boldsymbol{u}_i$ are considered for the minimization, if one ensures that

[8] The state space $\boldsymbol{x}$ has the dimension of twice the number of joint degrees of freedoms n since it includes also velocities. The dimension of $\boldsymbol{u}$ is equal to the number of joints n.

the states are function of the control variables through a corresponding integration of the dynamics, namely

$$\begin{aligned} \min_{\boldsymbol{u}} \quad & \Phi \simeq \sum_{i=0}^{n_t-1} \frac{\Delta t}{2}\Big[\mathcal{C}\big(\boldsymbol{x}_i(\boldsymbol{u}),\boldsymbol{u}_i\big)+\mathcal{C}\big(\boldsymbol{x}_{i+1}(\boldsymbol{u}),\boldsymbol{u}_{i+1}\big)\Big] \\ & \boldsymbol{x}_{i=n_t}(\boldsymbol{u}) = \boldsymbol{x}_e \\ & |\boldsymbol{x}_i(\boldsymbol{u})| \leq \boldsymbol{x}_{max} \;\; ; \;\; |\boldsymbol{u}_i| \leq \boldsymbol{u}_{max} \qquad \forall i = 1,\dots n_t \end{aligned} \tag{4.4.31}$$

where the notation $\boldsymbol{x}_i(\boldsymbol{u})$ indicates that the states are functions of the time history of the control variables. $\boldsymbol{x}_i(\boldsymbol{u})$ is then computed from the forward dynamics through a time integration starting from $\boldsymbol{x}_0$, applying for instance section 4.4.6 or some cheaper explicit Euler forward technique.[9] Such an approach is called a "single shooting method", where the trajectory of $\boldsymbol{x}$ is computed in time for the estimates $\boldsymbol{u}_i$ successively generated during the solution process of the optimal control problem (for instance with an SQP, see next). Note that the advantage of writing the NLP like in eq. (4.4.31) compared to eq. (4.4.30) is that there are much less optimization variables to find (but the complexity of the NLP is increased).

- Similarly, the minimization problem could be transformed in an optimization problem for $\boldsymbol{x}_i$ only, the control variables being assumed to be functions of the states through the inverse dynamics, namely

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & \Phi \simeq \sum_{i=0}^{n_t-1} \frac{\Delta t}{2}\Big[\mathcal{C}\big(\boldsymbol{x}_i,\boldsymbol{u}_i(\boldsymbol{x})\big)+\mathcal{C}\big(\boldsymbol{x}_{i+1},\boldsymbol{u}_{i+1}(\boldsymbol{x})\big)\Big] \\ & \boldsymbol{x}_{i=0} = \boldsymbol{x}_0 \;\; ; \;\; \boldsymbol{x}_{i=n_t} = \boldsymbol{x}_e \\ & |\boldsymbol{x}_i| \leq \boldsymbol{x}_{max} \;\; ; \;\; |\boldsymbol{u}_i(\boldsymbol{x}_i)| \leq \boldsymbol{u}_{max} \qquad \forall i = 1,\dots n_t \end{aligned} \tag{4.4.32}$$

where $\boldsymbol{u}_i(\boldsymbol{x}_r)$ indicates that the control variables are obtained from the inverse dynamics of the robot, which is typicall much cheaper to compute then the direct dynamics (see Chapter 3).

Solution of the NLP by SQP NLP problems like eq. (4.4.30), eq. (4.4.31) or eq. (4.4.32) can be put in the general form

$$\boxed{\begin{aligned} \min_{\boldsymbol{z}} \quad & \Phi(\boldsymbol{z}) \\ & \boldsymbol{h}(\boldsymbol{z}) = \boldsymbol{0} \\ & \boldsymbol{g}(\boldsymbol{z}) \leq \boldsymbol{0} \end{aligned}} \tag{4.4.33}$$

For eq. (4.4.30), one would have $\boldsymbol{z} = [\boldsymbol{x}_0 \cdots \boldsymbol{x}_{n_t}; \boldsymbol{u}_0 \cdots \boldsymbol{u}_{n_t}]$. In the Sequential Quadratic Programming (SQP) approach, one searches the optimal variables $\boldsymbol{z}$ iteratively. Let us

[9] One has to realize that solving section 4.4.6 is quite expensive. Indeed, in a time-stepping scheme, one applies section 4.4.6 to compute $\boldsymbol{x}_{i+1}$ from $\boldsymbol{x}_i$. Since section 4.4.6 is a nonlinear and implicit equation for $\boldsymbol{x}_{i+1}$, solving it is expensive. So a simple Euler forward integration, namely

$$\boldsymbol{x}_{i+1} = \boldsymbol{x}_i + \boldsymbol{a}(\boldsymbol{x}_i,\boldsymbol{u}_i)\Delta t$$

is much cheaper, although less accurate and possibly even unstable.

call $z^{(k)}$ the estimate of the optimal variable at iteration k of the SQP (the first estimate being obtained for instance from a sampling-based strategy). The next estimate is then computed as

$$z^{(k+1)} = z^{(k)} + \delta^{(k)} \tag{4.4.34}$$

where $\delta^{(k)}$ is the correction of the current estimate. One then approximates the cost function by a quadratic form and the contraints by a linearized form:

$$\begin{aligned} \min_{\delta^{(k)}} \quad & \Phi(z^{(k)} + \delta^{(k)}) \simeq \Phi(z^{(k)}) + \delta^{(k)T} \frac{d\Phi(z^{(k)})}{dz} + \frac{1}{2}\delta^{(k)T} \frac{d^2\Phi(z^{(k)})}{dz^2}\delta^{(k)} \\ & h(z^{(k)}) + \frac{dh(z^{(k)})}{dz}\delta^{(k)} = 0 \\ & g(z^{(k)}) + \frac{dg(z^{(k)})}{dz}\delta^{(k)} \leq 0 \end{aligned} \tag{4.4.35}$$

Similarly to how we obtained eq. (4.4.13) and eq. (4.4.17), the constrained minimum of the Quadratic Problem section 4.4.6 leads to the problem

$$\begin{bmatrix} \frac{d^2\Phi(z^{(k)})}{dz^2} & \frac{dh(z^{(k)})}{dz}^T & \frac{dg(z^{(k)})}{dz}^T \\ \frac{dh(z^{(k)})}{dz} & 0 & 0 \\ \frac{dg(z^{(k)})}{dz} & 0 & 0 \end{bmatrix} \begin{bmatrix} \delta^{(k)} \\ \lambda^{(k)} \\ \mu^{(k)} \end{bmatrix} = \begin{bmatrix} -\frac{d\Phi(z^{(k)})}{dz} \\ -h(z^{(k)}) \\ -g(z^{(k)}) \end{bmatrix} \tag{4.4.36}$$

where the first line is in fact the Euler-Lagrange equation of the NLP and the other lines arise from the linearized constrained problem. Remember that for each inequality constraint one has to satisfy the KKT conditions (see section 4.4.3)

$$g_j(z^{(k)} + \delta^{(k)}) \leq 0 \tag{4.4.37}$$

$$g_j(z^{(k)} + \delta^{(k)})\ \mu_j^{(k)} = 0 \tag{4.4.38}$$

$$\mu_j^{(k)} \geq 0 \tag{4.4.39}$$

which have to be checked at every iteration and decide of they are active or not. These equations can be solved numerically for $\delta^{(k)}$ in order to compute the update of the estimates of the optimization variables.

Remarks :

- In practice, especially in presence of inequalities and when the NLP is very nonlinear, the update of the optimal variable during the SQP iterations is performed in a more conservative manner than section 4.4.6 in order to limit the modification of the estimate and avoid drifting far away from the true solution of the SQP:

  $$z^{(k+1)} = z^{(k)} + \alpha\delta^{(k)} \tag{4.4.40}$$

  This is called a line-search, since one evolves the solution along the direction ("line") $\delta^{(k)}$, but does not perform the full update by choosing $\alpha \in [0, 1]$ depending on some heuristics, choosing a small α when the minimum conditions are far from being met, and $\alpha = 1$ when close to the optimum.

- Other techniques exist to find the solution of the discretized optimal problem section 4.4.6. But in all effective techniques, the cost to set up the problem (like in

section 4.4.6 for the SQP) is significant. It requires obtaining the gradients of the constraint and of the cost function with respect to z. Even the Hessian (second derivative) of the cost function is necessary in section 4.4.6. For robotic problems, we have to remember that the equality conditions involve the dynamics (or the kinematics) of the robot (see for instance section 4.4.6), which makes the computation of the required gradients cumbersome. Also conditions such as avoiding collision involve cumbersome relations between states and expensive evaluation of distances to obstacles. Collision conditions can enter in the minimization function (penalization) or as inequalities, but in any case result in intricate contributions in the minimization problem.

- The gradients needed for solving the NLP can sometimes be obtained analytically (if one has an analytical description of the dynamics and of the collision for instance), but can also be evaluate by finite difference. In that case, the constraints and the dynamics need to be evaluated for several state and control values, which is usually very expensive. In summary, one has to be aware of the complexity and the cost involved when applying optimal control.

- Many codes exist to solve NLP problems. The function *fmincon* in Matlab® is an example of solver. Other codes are for instance IPOPT [9] or qpOASES [1]. Be aware that when using such tools, one still has to provide the routines to compute the constraint and minimization functions and, possibly, the gradients.

4.4.7 Indirect Optimization

In order to reformulate the optimal control problem in a variational form, one can proceed as in the time-local optimization case (section 4.4.2) and introduce a Lagrange multipliers $\boldsymbol{\lambda}(t)$ to force the minimum to stay on the constraints over the entire trajectory. Similarly to eq. (4.4.12), we define the Lagrangian and the related stationarity condition that is equivalent to the constrained optimization :

$$\begin{aligned} &\mathcal{L}\big(\boldsymbol{x}(t),\boldsymbol{u}(t),\boldsymbol{\lambda}(t)\big) \stackrel{\text{def}}{=} C(\boldsymbol{x},\boldsymbol{u}) + \big(\boldsymbol{a}(\boldsymbol{x},\boldsymbol{u}) - \dot{\boldsymbol{x}}\big)^T \boldsymbol{\lambda} \\ &\delta_{\boldsymbol{x},\boldsymbol{u},\boldsymbol{\lambda},\boldsymbol{\nu}} \left\{ \int_{t_0}^{t_e} \mathcal{L}\big(\boldsymbol{x}^*(t),\boldsymbol{u}^*(t),\boldsymbol{\lambda}^*(t)\big)\, dt + \boldsymbol{\nu}^{*T}(\boldsymbol{x}^*(t_e) - \boldsymbol{x}_e) \right\} = 0 \end{aligned} \tag{4.4.41}$$

where $\boldsymbol{x}(t)^*, \boldsymbol{u}(t)^*, \boldsymbol{\lambda}(t)^*$ represent the optimal functions over the trajectory. The Lagrange multipliers $\boldsymbol{\nu}$ have been introduced to impose the terminal condition $\boldsymbol{x}(t_e) = \boldsymbol{x}_e$. they are not time functions, but constants to be determined. It is important to understand here, that the variations are virtual changes of *functions* (remember that the function to minimize is a functional, i.e. a function of functions). This is schematically illustrated (for instance for $\boldsymbol{x}$) in fig. 4.19. From the illustration, it is clear that a variation $\delta\boldsymbol{x}$ implies also a change of the variation $\delta\dot{\boldsymbol{x}}$.

Let us then perform the variation to discover the equations that the optimum trajectory needs to satisfy (similar to the Euler-Lagrange equation found for time-local optimization in section 4.4.2).

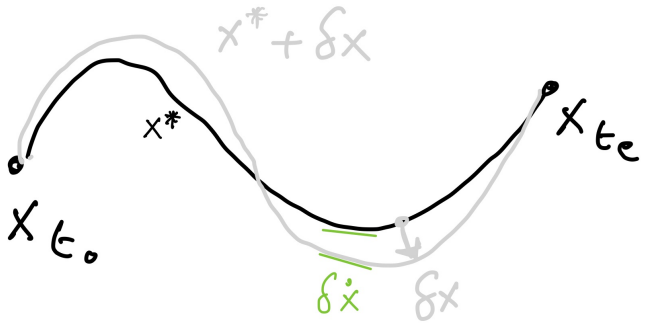


Figure 4.19: Illustration of a virtual variation around the optimum trajectory of the states

Variation $\delta \boldsymbol{x}$

$$\int_{t_0}^{t_e} \left(\delta \boldsymbol{x}^T \frac{\partial \mathcal{C}}{\partial \boldsymbol{x}} + \delta \boldsymbol{x}^T \frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{x}} \boldsymbol{\lambda} - \delta \dot{\boldsymbol{x}}^T \boldsymbol{\lambda} \right) dt + \delta \boldsymbol{x}(t_e)^T \boldsymbol{\nu} = 0 \tag{4.4.42}$$

Here we have to realize that $\delta \boldsymbol{x}$ and $\delta \dot{\boldsymbol{x}}$ are not independent since a variation of $\boldsymbol{x}(t)$ implicitly changes $\dot{\boldsymbol{x}}$ (see also fig. 4.19). Using an integration by part,

$$\int_{t_0}^{t_e} \delta \dot{\boldsymbol{x}}^T \boldsymbol{\lambda} \, dt = \left[\delta \boldsymbol{x}^T \boldsymbol{\lambda} \right]_{t_0}^{t_e} - \int_{t_0}^{t_e} \delta \boldsymbol{x}^T \dot{\boldsymbol{\lambda}}(t_e) \, dt \tag{4.4.43}$$

$$= \delta \boldsymbol{x}(t_e)^T \boldsymbol{\lambda}(t_e) - \int_{t_0}^{t_e} \delta \boldsymbol{x}^T \dot{\boldsymbol{\lambda}} \, dt \tag{4.4.44}$$

since $\delta \boldsymbol{x}(t_0) = 0$, the initial condition being fixed. Substituting this relation in section 4.4.7 and grouping all the terms, one finally finds

$$\int_{t_0}^{t_e} \delta \boldsymbol{x}^T \left(\frac{\partial \mathcal{C}}{\partial \boldsymbol{x}} + \frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{x}} \boldsymbol{\lambda} + \dot{\boldsymbol{\lambda}} \right) dt + \delta \boldsymbol{x}(t_e)^T \left(\boldsymbol{\nu} - \boldsymbol{\lambda}(t_e) \right) = 0 \tag{4.4.45}$$

Since this must hold for ay arbitrary $\delta \boldsymbol{x}$, the optimum must satisfy

$$\frac{\partial \mathcal{C}}{\partial \boldsymbol{x}} + \frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{x}} \boldsymbol{\lambda} + \dot{\boldsymbol{\lambda}} = \boldsymbol{0} \qquad \text{for } t_0 < t < t_e \tag{4.4.46}$$

$$\boldsymbol{\nu} - \boldsymbol{\lambda}(t_e) = \boldsymbol{0} \tag{4.4.47}$$

Variation $\delta \boldsymbol{u}$

$$\int_{t_0}^{t_e} \left(\delta \boldsymbol{u}^T \frac{\partial \mathcal{C}}{\partial \boldsymbol{u}} + \delta \boldsymbol{u}^T \frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{u}} \boldsymbol{\lambda} \right) dt \tag{4.4.48}$$

which must hold for all $\delta \boldsymbol{u}$ and thus

$$\frac{\partial \mathcal{C}}{\partial \boldsymbol{u}} + \frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{u}} \boldsymbol{\lambda} = \boldsymbol{0} \qquad \text{for } t_0 < t < t_e \tag{4.4.49}$$

Variation δ_λ, δ_ν One can finally easily verify that the variation of the Lagrange multipliers respectively yields

$$\boldsymbol{a}(\boldsymbol{x},\boldsymbol{u})-\dot{\boldsymbol{x}}=\boldsymbol{0} \tag{4.4.50}$$
$$\boldsymbol{x}_e-\boldsymbol{x}(t_e)=\boldsymbol{0} \tag{4.4.51}$$

Summarizing, the stationarity expressed by eq. (4.4.41) results in the following equations that must be satisfied by the optimal solution:

$$\boxed{\begin{aligned}&\frac{\partial \mathcal{C}}{\partial \boldsymbol{x}}+\frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{x}}\boldsymbol{\lambda}+\dot{\boldsymbol{\lambda}}=\boldsymbol{0}\\&\frac{\partial \mathcal{C}}{\partial \boldsymbol{u}}+\frac{\partial \boldsymbol{a}^T}{\partial \boldsymbol{u}}\boldsymbol{\lambda}=\boldsymbol{0}\\&\boldsymbol{a}(\boldsymbol{x},\boldsymbol{u})-\dot{\boldsymbol{x}}=\boldsymbol{0}\\&\\&\boldsymbol{\nu}-\boldsymbol{\lambda}(t_e)=\boldsymbol{0}\\&\boldsymbol{x}(t_0)=\boldsymbol{x}_0\\&\boldsymbol{x}(t_e)=\boldsymbol{x}_e\end{aligned}} \tag{4.4.52}$$

The first three equations are partial differential equations (PDEs) that determine the time evolution of $\boldsymbol{x}(t),\boldsymbol{u}(t),\boldsymbol{\lambda}(t)$. We note that $\boldsymbol{x}(t),\boldsymbol{\lambda}(t)$ have to satisfy first-order differential equation in time and have the boundary conditions given by the last three equations.

Contrary to the direct optimization approach of section 4.4.6, here we have first deduced, from the stationarity property, the partial differential equations that must be satisfied by the solution, then the solution is (numerically) found for these PDEs. It is thus known as an **indirect optimization** approach. In other words, the direct approach "first discretizes then minimizes", whereas the indirect method first "minimizes" then discretizes.[10]

The PDEs section 4.4.7 can be solved with numerical techniques similar to what was discussed for the direct optimization technique in section 4.4.6. The specific numerical techniques for the indirect approach will not be further discussed here. However let us make the following remarks on the indirect approach:

Remarks

- In case no terminal condition $\boldsymbol{x}_e$ is imposed, $\boldsymbol{\nu}$ is simply set to zero.

[10] For those familiar with mechanical dynamics, the difference between direct and indirect approaches can be understood by considering the Hamilton principle which states that the solution over a trajectory must be the stationary point of the Action. For the simple example of a mass-spring system on a line,

$$\delta_{\boldsymbol{q}}\int_{t_0}^{t_e}\text{Action}\,dt=\delta_{\boldsymbol{q}}\int_{t_0}^{t_e}(\text{Kinetic Energy}-\text{Potential Energy})\,dt=\delta_q\int_{t_0}^{t_e}(\frac{1}{2}m\dot{q}^2-\frac{1}{2}kq^2)\,dt=0$$

The solution can be found by directly discretizing $\boldsymbol{x}_0$ in time and find a numerical solution by writing the stationarity of the discretized Action. Alternatively, one can first perform analytically the variation to get the dynamic equations

$$m\ddot{q}+kq=0$$

and then time-discretize this second-order problem to find the solution. Although both methods give theoretically the same answer, the approximate solution found by discretization can differ in quality.

- The optimal control considered for the indirect optimization in this section is a rather simplified one. In practice, the optimal control problem is more complicated, for instance
  - The end time t_e can be let free and be optimized together with the other variables.
  - An additional cost $\phi(t_e, \boldsymbol{x}_0(t_e))$ is often added to the stationarity eq. (4.4.41) in order for instance to minimize the duration $t_e - t_0$ or to force the trajectory to end close to an obstacle.
  - Equality and inequality constraints on $\boldsymbol{x}$ and $\boldsymbol{u}$ or often added to the optimal control problem eq. (4.4.41). These need to be handled by additional Lagrange multipliers or KKT conditions. One inequality constraint that is commonly considered for the optimal control problem eq. (4.4.20) is a limit on the control inputs $\boldsymbol{u}$ representing for instance limits on the motor moments (currents) in the form
    $$\boldsymbol{u}_{min} < \boldsymbol{u}(t) < \boldsymbol{u}_{max}$$
    When such an inequality exists, the variation in the stationarity problem as done in section 4.4.7 does not directly yield the condition section 4.4.7 since the variations are not arbitrary anymore (they need to satisfy the inequality above). In that case, different cases need to be considered (similar to what was done for the KKT conditions in section 4.4.3). This problem is handled by the Pontryagin's Maximum Principle (PMP). Note that, since $\boldsymbol{u}$ does not have to satisfy a time differential equation (see section 4.4.7), it can be discontinuous in time in order to satisfy these conditions, leading to so-called "bang-bang" controls where the controls switch from maximum to minimum or vice-versa.
- Analytical PDEs obtained in the indirect approach can be solved analytically for simple problems (for instance for rocket trajectories). For robots however, the number of variables and the complexity of the constraints (obstacles, joint limits ...) are usually such that a solution can be found only with specific numerical strategies. One particularity of solution techniques for section 4.4.7 is that the states $\boldsymbol{x}$ need to be integrated *forward* in time ($\boldsymbol{x}_e$ given), whereas the Lagrange multipliers are integrated *backward* in time since end conditions are specified for them.

Bibliography

[1] Ferreau, H. J., Kirches, C., Potschka, A., Bock, H. G., and Diehl, M. "qpOASES: A parametric active-set algorithm for quadratic programming". In: *Mathematical Programming Computation* 6.4 (2014), pp. 327–363.

[2] Kavraki, L., Svestka, P., Latombe, J.-C., and Overmars, M. "Probabilistic roadmaps for path planning in high-dimensional configuration spaces". In: *IEEE Transactions on Robotics and Automation* 12.4 (1996), pp. 566–580. DOI: 10.1109/70.508439.

[3] Kist, A. "Predicitive Kinematics Control for Motion Optimization of Manipulators". Masterarbeit. Lehrstuhl f. Angewandte Mechanik, 2021.

[4] Larsen, E., Gottschalk, S., Lin, M., and Manocha, D. "Fast distance queries with rectangular swept sphere volumes". In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*. Vol. 4. 2000, 3719–3726 vol.4. DOI: 10.1109/ROBOT.2000.845311.

[5] LaValle, S. *Rapidly-exploring random trees: A new tool for path planning*. 1998. URL: http://lavalle.pl/papers/Lav98c.pdf.

[6] LaValle, S. *Planning Algorithms*. Cambridge University Press, 2006. ISBN: 9781139455176. URL: http://lavalle.pl/planning/.

[7] Lohmeier, S. "Design and Realization of a Humanoid Robot for Fast and Autonomous Bipedal Locomotion". Dissertation. München: Technische Universität München, 2010. URL: http://nbn-resolving.de/urn/resolver.pl?urn:nbn:de:bvb:91-diss-20101126-980754-1-4.

[8] Schuetz, C., Buschmann, T., Baur, J., Pfaff, J., and Ulbrich, H. "Predictive online inverse kinematics for redundant manipulators". In: *2014 IEEE international conference on robotics and automation (ICRA)*. IEEE. 2014, pp. 5056–5061.

[9] Wächter, A. and Biegler, L. T. "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming". In: *Mathematical programming* 106.1 (2006), pp. 25–57.

[10] Wittmann, J., Kist, A., and Rixen, D. J. "Real-Time Predictive Kinematics Control of Redundancy: a Benchmark of Optimal Control Approaches". In: *International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2022.